Crowdbotics

Product Requirements Document

Amalia Litsa

Crowdbotics, Inc.

Table of contents

Application

Description

Consolidated List of Functional Features

Core Server Features

1. **Server Initialization and Configuration**: Prepares the server for operation by loading configuration files, setting up the environment, and validating settings. Allows customization through configuration files with error handling for invalid or missing values.
2. **Graceful Shutdown and Persistence**: Ensures a smooth server shutdown by saving game state, disconnecting clients, and releasing resources. Supports periodic autosaving to prevent data loss.
3. **Cross-Platform Deployment**: Enables platform-independent deployment using Docker and a build system that supports GnuCOBOL, C++, and Makefile.
4. **Version Compatibility**: Supports specific Minecraft versions with a structured process for updating to newer versions.

Client and Connection Management

5. **Client Connection Handling**: Manages client connections, including accepting new players, tracking their states, and handling disconnections or errors.
6. **Keep-Alive Mechanism**: Maintains active connections by sending periodic keep-alive packets and disconnecting unresponsive clients.
7. **Multiplayer Support**: Allows multiple players to connect and interact, with configurable player limits and whitelist management.
8. **Handshake and Login Management**: Validates client protocol versions, verifies usernames, and generates player UUIDs during connection initialization.

World and Chunk Management

9. **Infinite Terrain Generation**: Dynamically generates and loads terrain as players explore, ensuring seamless world expansion.
10. **Chunk Management**: Handles loading, saving, and unloading of chunks, optimizing memory usage and rendering by prioritizing nearby chunks.
11. **Spawn Area Management**: Keeps spawn area chunks loaded in memory for quick access.
12. **World Metadata Management**: Tracks and saves world-level attributes such as time, spawn point, and game mode settings.
13. **Block Manipulation**: Supports getting, setting, and updating blocks, including handling block entities and notifying clients of changes.
14. **Boundary Validation**: Ensures player and block positions remain within the world's defined vertical bounds.

Player Management

15. **Player Initialization and Persistence**: Initializes player data (e.g., position, health, inventory) and saves it to disk for continuity across sessions.
16. **Player Interaction**: Manages player actions such as breaking, placing, and interacting with blocks, as well as crafting and inventory management.
17. **Player Abilities and Game Modes**: Updates player abilities (e.g., flying, creative mode) based on game mode or commands.
18. **Player Respawn**: Handles player respawn events, resetting health, position, and inventory for seamless re-entry into the game.
19. **Collision Detection**: Ensures players do not collide with blocks, entities, or other players during interactions.

Inventory and Crafting

20. **Inventory Management**: Synchronizes player inventory with the client, supports item stacking, swapping, and storage, and handles item drops.
21. **Crafting System**: Provides 2x2 and 3x3 crafting grids, dynamically updates crafting outputs based on inputs, and supports custom recipes.
22. **Crafting Window Management**: Ensures proper synchronization of crafting windows, slot updates, and item drops.

Entity Management

23. **Entity Lifecycle Management**: Manages the spawning, updating, and removal of entities, ensuring unique IDs and notifying clients of changes.
24. **Entity Collision Detection**: Detects collisions between entities and other objects in the game world.
25. **Entity Metadata Synchronization**: Updates and synchronizes entity attributes (e.g., health, equipment) with clients.

Block and Item Features

26. **Block Registration and Behavior**: Defines block behaviors, such as interaction, destruction, and metadata configuration, including support for special blocks like doors, slabs, and beds.
27. **Item Interaction**: Implements item-specific behaviors, such as placing blocks, using buckets, and interacting with crafting tables.
28. **Fluid Placement**: Supports placing fluids like water and lava, ensuring valid placement positions.
29. **Custom Block Properties**: Configures block-specific properties (e.g., orientation, waterlogging) during placement.

Whitelist and Permissions

30. **Whitelist Management**: Manages a whitelist of allowed players, including adding, removing, and verifying players. Stores whitelist data persistently.
31. **Permission Handling**: Grants operator permissions to players for administrative tasks.

Command System

32. **Command Registration and Execution**: Supports server and gameplay commands (e.g., gamemode, save, stop, time, whitelist) for server management and player interaction.
33. **Command Autocompletion**: Provides clients with command structure and options for autocompletion.

Chat and Communication

34. **Chat System**: Enables in-game communication between players, with support for broadcasting messages and server console logging.
35. **Player Chat Messages**: Sends formatted chat messages to clients, including sender information.

Game State and Debugging

36. **Game Loop**: Executes the main game loop, updating the game state, broadcasting player positions, and processing interactions.
37. **Debug Profiling**: Collects and sends performance metrics (e.g., tick duration, idle time) to clients for debugging purposes.

Packet Management

38. **Packet Handling**: Manages incoming and outgoing packets, including parsing, validation, and sending updates to clients.
39. **Game State Synchronization**: Ensures changes made by players (e.g., placing blocks, using items) are accurately reflected in the game world and communicated to other players.

Data Serialization and Parsing

40. **NBT Encoding and Decoding**: Encodes and decodes structured data (e.g., blocks, biomes, entities) in the NBT format for efficient storage and transmission.
41. **JSON Parsing and Encoding**: Provides tools for parsing and generating JSON data, including crafting recipes and configuration files.
42. **Recipe Parsing**: Validates and processes crafting recipes (shaped and shapeless) from JSON input.

File and Directory Management

43. **Region File Handling**: Manages region files for storing chunk data, including compression, decompression, and sector allocation.
44. **File Operations and Error Handling**: Ensures proper file creation, reading, and writing, with robust error handling to maintain data integrity.

Physics and Collision

45. **Axis-Aligned Bounding Box (AABB) Collision**: Detects collisions between objects in 3D space for accurate interactions.
46. **Player Bounding Box**: Calculates player bounding boxes, accounting for states like sneaking.

Utility Features

47. **UUID Management**: Converts UUIDs between string and binary formats for use in player and entity management.
48. **Hexadecimal Encoding/Decoding**: Provides utilities for encoding and decoding hexadecimal strings.
49. **Compression and Decompression**: Supports gzip and zlib compression for efficient data storage and transmission.
50. **Dynamic Memory Management**: Optimizes resource usage by allocating and deallocating memory for chunks, entities, and other game data.

Connection and Session Management

51. **Client Configuration**: Processes client settings (e.g., locale, view distance, chat preferences) and sends server configuration packets in response.
52. **Ping and Latency Monitoring**: Monitors connection health by responding to client pings and measuring latency.
53. **Server Status Reporting**: Provides server information, including version, player count, and a message of the day (MOTD), in response to client requests.

Server Infrastructure and Build Management

54. **Multi-Stage Docker Build**: Optimizes containerized deployment by separating build and runtime stages.
55. **Build and Dependency Management**: Compiles source files and manages runtime dependencies for efficient application execution.
56. **Signal Handling**: Ensures smooth operation in containerized environments by properly handling process signals.
57. **Test Support**: Includes functionality to compile and run tests, ensuring application reliability.

This consolidated list captures the application's comprehensive functionality, emphasizing its capabilities for managing a robust, multiplayer game server environment.

User Types

- Server Administrators
- Players
- Developers

Phase 1

Analytics & Monitoring

Precise System Time Retrieval

Description

The 'Precise System Time Retrieval' feature in the 'Litsa Minecraft Cobol' application provides highly accurate system timestamps in both microseconds and milliseconds. This feature is essential for time-sensitive operations such as logging, performance monitoring, and event scheduling. By leveraging the system clock, it retrieves the current time and converts it into the desired precision format. The feature is seamlessly integrated into the application's core systems, ensuring that developers and administrators can utilize precise time data for debugging, analytics, and optimizing server performance. This functionality is particularly useful for tracking event sequences, measuring execution times, and maintaining synchronization across distributed systems.

Acceptance Criteria

- The feature must retrieve the current system time with precision up to microseconds and milliseconds.
- The retrieved timestamps must be accurate and consistent with the system clock, ensuring no significant drift or deviation.
- The feature must provide a simple and efficient API or utility function for developers to access the precise timestamps.
- The timestamps must be formatted in a standard and easily interpretable structure, such as ISO 8601 or Unix epoch time, with clear differentiation between microseconds and milliseconds.
- The feature must integrate seamlessly with existing logging systems, allowing timestamps to be appended to log entries without additional configuration.
- The system time retrieval process must demonstrate high performance, with minimal overhead or latency introduced during execution.
- The feature must support both synchronous and asynchronous workflows, ensuring compatibility with various use cases, including real-time event tracking and batch processing.
- The implementation must include robust error handling to manage potential issues, such as system clock failures or invalid time formats, and provide meaningful feedback to the user or developer.
- The feature must be tested across different operating systems and environments to ensure consistent behavior and compatibility.
- The system time retrieval functionality must be documented thoroughly, including usage examples, API references, and integration guidelines for developers.
- The feature must include monitoring capabilities to track its usage and performance, enabling further optimization if necessary.
- The timestamps generated by the feature must be compatible with other components of the application, such as analytics modules, event schedulers, and debugging tools.

Testing Framework for Utility Functions

Description

The 'Testing Framework for Utility Functions' feature in the 'Litsa Minecraft Cobol' application ensures the correctness and reliability of core utility functions, such as compression, decompression, and data decoding. This framework provides a comprehensive suite of test cases designed to validate the functionality of these utilities under various scenarios. By automating the testing process, it helps developers identify and address potential issues early in the development cycle, reducing the risk of bugs in production. The framework is modular and extensible, allowing for the addition of new test cases as the application evolves. It also generates detailed reports on test results, enabling developers and QA engineers to track the performance and reliability of utility functions over time. This feature is critical for maintaining the stability and robustness of the application, especially in areas involving data processing and serialization.

Acceptance Criteria

- The testing framework must include a comprehensive suite of test cases covering all core utility functions, including compression, decompression, and data decoding.
- The framework must support automated execution of test cases, with the ability to run tests individually or as a batch.
- Test cases must validate the functionality of utility functions under normal, edge, and error conditions, ensuring robust performance across all scenarios.
- The framework must generate detailed test reports, including pass/fail status, execution time, and error details for failed tests.
- The framework must be modular and extensible, allowing developers to easily add new test cases for additional utility functions or new features.
- The testing process must integrate seamlessly with the application's build and deployment pipeline, ensuring that tests are executed as part of the CI/CD workflow.
- The framework must include error handling to gracefully manage unexpected issues during test execution, providing meaningful feedback to developers.
- The framework must support cross-platform compatibility, ensuring that tests can be executed consistently across different environments and operating systems.

- The framework must provide clear documentation and examples to help developers and QA engineers understand how to use and extend it effectively.
- The framework must demonstrate high performance, executing tests quickly without introducing significant delays to the development or deployment process.

Debug Information Subscription

Description

The 'Debug Information Subscription' feature in the 'Litsa Minecraft Cobol' server allows operators, developers, and system administrators to subscribe to real-time debug information for monitoring and troubleshooting purposes. This feature provides detailed insights into server operations, such as game tick updates, player activity, and system performance metrics. Subscriptions can be initiated via server commands or integrated monitoring tools, enabling users to receive relevant debug data in a structured format. The server processes these subscription requests and streams the requested information efficiently, ensuring minimal performance impact. This feature is designed to integrate seamlessly with external server monitoring tools, providing a robust mechanism for diagnosing issues and optimizing server performance.

Acceptance Criteria

- The server must support a command or API endpoint for initiating debug information subscriptions, allowing users to specify the type of data they wish to receive (e.g., game tick updates, player activity, system performance metrics).
- The debug information must be streamed in real-time to the subscribing user or tool, ensuring minimal latency and accurate monitoring.
- The feature must integrate with external server monitoring tools, providing data in a standardized format (e.g., JSON or plain text) for easy parsing and analysis.
- The server must allow multiple concurrent subscriptions without significant performance degradation, ensuring scalability for large-scale monitoring.
- The subscription system must include configurable filters, enabling users to narrow down the debug data to specific categories or events of interest.
- The server must log all subscription requests and their corresponding data streams for auditing and troubleshooting purposes.
- The feature must include error handling to manage invalid subscription requests, such as unsupported data types or malformed commands, providing clear feedback to the user.
- The debug information subscription must be secure, ensuring that only authorized users (e.g., server operators or administrators) can access sensitive debug data.
- The server must provide a mechanism to unsubscribe from debug information streams, either through a command or an API endpoint, ensuring users can stop receiving data when no longer needed.
- The feature must be tested across different server configurations and environments to ensure consistent behavior and compatibility.
- The debug information must be formatted for readability and include timestamps, making it easier for users to correlate events and analyze server behavior.
- The system must include safeguards to prevent excessive resource usage from debug subscriptions, such as rate limiting or prioritizing critical server operations over debug data streaming.

User Interaction

Non-Blocking Console Input

Description

The 'Non-Blocking Console Input' feature in the 'Litsa Minecraft Cobol' application enables asynchronous console input, allowing the application to process user input without interrupting or halting other ongoing operations. This feature is particularly useful in interactive applications or games where real-time input is required, such as during gameplay or server management. It works by configuring the console (stdin) to non-blocking mode using system calls, ensuring that the application can continue executing other tasks while monitoring for user input. This implementation enhances responsiveness and ensures smooth operation in scenarios where simultaneous input and processing are critical.

Acceptance Criteria

- The console (stdin) must be successfully configured to non-blocking mode using appropriate system calls, ensuring that input operations do not block the main application thread.
- The application must be able to detect and process user input asynchronously, without causing delays or interruptions to other ongoing operations.
- In cases where no input is provided, the application must continue executing other tasks without waiting or freezing.
- The feature must handle edge cases, such as invalid or unexpected input, gracefully without crashing or causing undefined behavior.
- The implementation must include error handling to manage potential issues with system calls or unsupported environments, providing clear feedback or fallbacks where necessary.
- The feature must be compatible with the supported platforms and operating systems of the application, ensuring consistent behavior across all environments.

- The non-blocking input functionality must be tested under various load conditions to ensure it performs reliably and does not introduce performance bottlenecks.
- The feature must include documentation for developers, explaining how the non-blocking input is implemented, how to use it, and any limitations or considerations.
- The application must log relevant events or errors related to non-blocking input for debugging and monitoring purposes.
- The feature must integrate seamlessly with other components of the application, such as command handling or gameplay mechanics, ensuring a cohesive user experience.

Content Management

Directory Management

Description

The 'Directory Management' feature in the 'Litsa Minecraft Cobol' application provides robust functionality for navigating and managing the file system. It enables developers and system administrators to efficiently interact with directories by offering methods to verify if a given path is a directory, open directory handles, read directory entries (excluding '.' and '..'), and close directory handles. This feature is essential for file management modules or applications that require directory traversal, ensuring seamless integration with the file system. By abstracting low-level directory operations, it simplifies the workflow for managing directories and enhances the reliability of file system interactions.

Acceptance Criteria

- The feature must provide a method to verify if a given path is a directory, returning a boolean value indicating the result.
- The feature must allow opening a directory handle for valid directory paths, with appropriate error handling for invalid paths or permissions issues.
- The feature must support reading directory entries, excluding special entries like '.' and '..', and return a list of valid directory contents.
- The feature must ensure that directory handles are properly closed after operations, preventing resource leaks.
- The feature must handle edge cases, such as empty directories, directories with a large number of entries, and directories with non-standard characters in their names.
- Error handling must be implemented to manage scenarios such as invalid paths, permission errors, or file system issues, providing meaningful error messages or codes.
- The feature must be compatible with the application's existing file management modules and integrate seamlessly into workflows requiring directory traversal.
- Performance must be optimized to handle directory operations efficiently, even for directories with a large number of entries.
- The feature must be tested across different operating systems and file systems to ensure consistent behavior and compatibility.
- The implementation must adhere to best practices for file system security, ensuring that directory operations do not expose sensitive information or allow unauthorized access.

Block Parsing and Management

Description

The 'Block Parsing and Management' feature in the 'Litsa Minecraft Cobol' application provides a robust system for defining, managing, and retrieving block properties and states. This feature is essential for enabling dynamic content creation and interaction within the game world. It parses a `blocks.json` file to populate a shared data structure with block definitions, properties, and states. Developers can utilize this feature to retrieve block state IDs, descriptions, and default states efficiently. Additionally, it offers utilities for iterating over block names and types, making it easier to manage and extend block-related functionality. This feature ensures consistency and scalability in handling block data, supporting both core gameplay mechanics and custom content creation.

Acceptance Criteria

- The system must successfully parse a `blocks.json` file containing block definitions, properties, and states, and populate a shared data structure with this information.
- The shared data structure must support efficient retrieval of block state IDs, descriptions, and default states.
- The feature must provide utilities for iterating over block names and types, enabling developers to access and manipulate block data programmatically.
- The parsing process must include error handling to validate the structure and content of the `blocks.json` file, ensuring that invalid or incomplete data does not disrupt the application.
- The feature must support dynamic updates to the `blocks.json` file, allowing new blocks or modifications to existing blocks to be reflected without requiring a full server restart.
- The system must ensure that block properties and states are consistent across all game components, including world generation, player interactions, and rendering.
- The feature must include logging for the parsing process, providing clear feedback on successful operations or errors encountered during parsing.
- The utilities provided must be optimized for performance, ensuring that block data retrieval and iteration do not introduce significant delays, even with large datasets.
- The feature must be compatible with the application's modular design, allowing it to integrate seamlessly with other systems such as chunk management, entity interaction, and crafting.
- The implementation must adhere to best practices for memory management, ensuring that the shared data structure does not lead to memory leaks or excessive resource usage.
- The feature must include unit tests to validate the correctness of block parsing, data retrieval, and utility functions, ensuring reliability and maintainability.

Region-Based World Storage

Description

The 'Region-Based World Storage' feature in the 'Litsa Minecraft Cobol' application optimizes the storage and retrieval of world data by dividing the game world into manageable regions. This approach enhances performance and scalability, especially for large worlds with extensive player activity. The system generates file names for region files based on their coordinates, ensuring a logical and consistent structure for data organization. It seamlessly integrates with player data management and game state updates, allowing for smooth interactions with the world. By leveraging this feature, the server can efficiently load, save, and manage chunks within specific regions, reducing memory usage and improving overall server responsiveness. This feature is critical for maintaining a scalable and performant multiplayer environment.

Acceptance Criteria

- The system must divide the game world into regions, with each region containing a fixed number of chunks (e.g., 32x32 chunks).
- Region files must be named based on their coordinates (e.g., 'r.x.z.mca'), ensuring compatibility with existing region-based world storage systems.
- The feature must support efficient loading and saving of region files, minimizing disk I/O operations and ensuring quick access to world data.
- Region-based storage must integrate seamlessly with player data management, ensuring that player interactions (e.g., block placement, movement) are accurately reflected in the corresponding region files.
- The system must handle game state updates, such as time progression and entity movements, without causing inconsistencies in region data.
- Regions that are not actively in use must be unloaded from memory to optimize resource usage, with the ability to reload them on demand.
- The feature must include error handling for corrupted or missing region files, providing fallback mechanisms to maintain server stability.
- Region-based storage must be compatible with the server's chunk management system, ensuring that chunks within a region are loaded and unloaded efficiently based on player proximity.
- The system must support backward compatibility with existing world data formats, allowing for seamless migration to the region-based storage system.
- Performance metrics, such as region file access times and memory usage, must be monitored and logged for debugging and optimization purposes.
- The feature must be tested across different server configurations and world sizes to ensure consistent performance and reliability.
- The implementation must adhere to best practices for data serialization and compression, ensuring that region files are stored efficiently without compromising data integrity.

Registry Management

Description

The 'Registry Management' feature in the 'Litsa Minecraft Cobol' application provides a robust and structured system for managing registries and their entries. This feature is designed to streamline the handling of data structures by enabling efficient querying, creation, and iteration of registry entries. It parses JSON files to load registries into shared data structures, ensuring seamless integration with other server components. Developers can create new registries, query existing ones by name or ID, and iterate over entries for advanced operations. This feature is essential for maintaining a modular and extensible server architecture, allowing for dynamic updates and efficient data management.

Acceptance Criteria

- The system must support parsing JSON files to load registries into shared data structures, ensuring compatibility with existing server components.
- Users must be able to create new registries dynamically, with proper validation to prevent duplicate or invalid entries.
- The feature must allow querying of registries by both name and ID, returning accurate and relevant results.
- Iterative access to registry entries must be supported, enabling developers to loop through entries for advanced operations or integrations.
- The registry management system must integrate seamlessly with other server components, such as block and item management, ensuring consistent data flow.
- Error handling must be implemented to manage invalid JSON files, duplicate entries, or other potential issues, providing clear feedback to the user.
- The system must demonstrate high performance, even with large datasets, ensuring minimal impact on server operations.
- Registry data must be stored persistently, ensuring that changes are saved and loaded correctly across server restarts.
- The feature must include a mechanism for updating or removing existing registry entries, maintaining data integrity and flexibility.
- Comprehensive logging must be implemented to track registry operations, such as creation, updates, and queries, for debugging and auditing purposes.
- The registry management system must be compatible with the latest version of the server and tested across different configurations to ensure reliability.

- The feature must adhere to security best practices, ensuring that registry data is protected from unauthorized access or tampering.

LRU-Based Resource Management

Description

The 'LRU-Based Resource Management' feature in the 'Litsa Minecraft Cobol' application ensures efficient memory and file handle management by implementing a Least Recently Used (LRU) strategy for region file handling. This feature monitors access times for region files and automatically closes the least recently used file when the system reaches its limit of open file slots. By doing so, it prevents resource exhaustion and maintains optimal system performance. The feature works seamlessly with the region file management system, ensuring that frequently accessed files remain available while less critical files are closed to free up resources. This approach is particularly beneficial in large-scale multiplayer environments where numerous region files are accessed simultaneously, as it minimizes latency and ensures smooth gameplay.

Acceptance Criteria

- The system must track access times for all open region files and maintain an internal priority list based on their usage frequency.
- When the maximum number of open file slots is reached, the system must automatically identify and close the least recently used region file to free up resources.
- Closed region files must be safely persisted to disk to ensure no data loss occurs during the closure process.
- The feature must integrate seamlessly with the existing region file management system, ensuring that closed files can be reopened when accessed again without errors or delays.
- The LRU mechanism must operate with minimal performance overhead, ensuring that the monitoring and file closure processes do not negatively impact server performance.
- The system must log all LRU-based file closure events, including the file name, closure timestamp, and reason for closure, for debugging and auditing purposes.
- The feature must handle edge cases, such as when all region files are actively in use, by prioritizing files with the least critical activity or lowest priority.
- The implementation must be compatible with the server's cross-platform deployment environment, including Docker and GnuCOBOL, ensuring consistent behavior across all supported platforms.
- The feature must include robust error handling to manage scenarios such as file access errors, disk write failures, or unexpected system interruptions, ensuring stability and reliability.
- The LRU-based resource management system must be thoroughly tested under various load conditions, including high player activity and large-scale world exploration, to validate its effectiveness and performance.

Biome and Block State Management

Description

The 'Biome and Block State Management' feature in the 'Litsa Minecraft Cobol' server ensures a consistent and optimized game environment by managing biomes and block states during terrain generation. When new chunks are initialized, this feature assigns default biomes (e.g., plains) to ensure a seamless and predictable starting point for terrain generation. It leverages palette-based storage to optimize memory usage and improve performance by efficiently encoding and decoding block and biome data. This feature integrates directly with the chunk generation and saving processes, ensuring that biome and block state data remain consistent and synchronized across the game world. By maintaining a structured approach to biome and block state management, this feature supports the server's ability to handle large-scale terrain generation while minimizing resource consumption.

Acceptance Criteria

- The feature must initialize new chunks with a default biome (e.g., plains) to ensure a consistent starting point for terrain generation.
- Biomes and block states must be stored using palette-based encoding to optimize memory usage and improve performance.
- The feature must integrate seamlessly with the chunk generation process, ensuring that biome and block state data are correctly assigned during chunk creation.
- Biome and block state data must be saved and loaded accurately during chunk saving and loading operations, ensuring data consistency across server restarts.
- The feature must support dynamic updates to block states and biomes, allowing for modifications during gameplay (e.g., biome changes or block updates).
- The system must handle edge cases, such as invalid biome or block state data, by falling back to default values or providing error handling mechanisms.
- Performance benchmarks must demonstrate that the feature does not introduce significant delays during chunk generation, saving, or loading processes.
- The feature must be compatible with the server's existing terrain generation and chunk management systems, ensuring no conflicts or regressions.
- The implementation must include unit tests and integration tests to validate the correctness of biome and block state assignments, as well as their storage and retrieval.
- The feature must provide debugging tools or logs to help developers and server administrators diagnose issues related to biome and block state management.

Linked List Management for Entities

Description

The 'Linked List Management for Entities' feature in the 'Litsa Minecraft Cobol' application provides an efficient mechanism for managing entities within chunks using a linked list data structure. This feature ensures seamless addition, removal, and traversal of entities, optimizing memory usage and performance. Each chunk maintains its own linked list of entities, allowing for dynamic interactions and efficient updates. The system integrates tightly with the entity management and chunk management subsystems, ensuring that entity operations such as spawning, despawning, and updates are handled in a structured and performant manner. This feature is critical for maintaining the integrity and responsiveness of the game world, especially in multiplayer environments where entity interactions are frequent and complex.

Acceptance Criteria

- Each chunk must maintain a dedicated linked list structure to store and manage entities within its boundaries.
- The linked list implementation must support efficient addition of new entities to the list, ensuring minimal performance overhead.
- Entities must be removable from the linked list without disrupting the integrity of the list or causing memory leaks.
- Traversal of the linked list must allow for efficient iteration over all entities in a chunk, enabling operations such as updates, rendering, and collision detection.
- The linked list must integrate seamlessly with the entity management system, ensuring that global entity operations (e.g., spawning, despawning) are reflected in the chunk-level linked lists.
- The linked list must also integrate with the chunk management system, ensuring that entities are properly loaded and unloaded as chunks are loaded or unloaded.
- The system must handle edge cases, such as attempting to remove an entity that does not exist in the list, without causing crashes or undefined behavior.
- Performance benchmarks must demonstrate that the linked list implementation can handle a high volume of entities per chunk without significant degradation in server performance.
- The feature must include robust error handling to manage scenarios such as invalid entity references or corrupted list structures.
- Unit tests must be implemented to validate the correctness of the linked list operations, including addition, removal, and traversal.
- The feature must be compatible with the existing entity and chunk management systems, requiring no major refactoring of these subsystems.
- Documentation must be provided to explain the linked list structure, its integration points, and best practices for extending or modifying the feature.

Data Decoding Utilities

Description

The 'Data Decoding Utilities' feature in the 'Litsa Minecraft Cobol' application provides a robust and efficient mechanism for decoding various data types from binary buffers. This feature is foundational for the application's data handling processes, ensuring accurate interpretation and conversion of raw binary data into usable formats such as bytes, integers, floats, strings, and 3D positions. It plays a critical role in supporting modules like entity management, world state updates, and network communication. By abstracting the complexities of binary data decoding, this utility ensures consistency, reliability, and maintainability across the application's codebase. Developers can leverage this feature to seamlessly process incoming data streams, enabling smooth gameplay experiences and efficient server operations.

Acceptance Criteria

- The utility must support decoding of multiple data types, including bytes, integers (signed and unsigned), floats, strings, and 3D positions, ensuring compatibility with the application's data structures.
- The decoding process must handle variable-length data formats, such as variable-length integers, without errors or data loss.
- The utility must provide clear error messages and robust error handling for scenarios where the binary buffer is malformed, incomplete, or contains unexpected data.
- The decoded data must be validated to ensure it adheres to the expected format and range, preventing invalid or corrupted data from propagating through the system.
- The feature must integrate seamlessly with other modules, such as entity management and world state updates, providing decoded data in a format that is immediately usable by these systems.
- Performance benchmarks must demonstrate that the decoding process operates with minimal latency, even when handling large or complex binary buffers, ensuring real-time responsiveness in gameplay scenarios.
- The utility must include comprehensive unit tests to validate its functionality across a wide range of input scenarios, including edge cases and stress tests.
- The feature must be documented with clear usage instructions and examples, enabling developers to quickly understand and implement it in their workflows.
- The utility must be compatible with the application's existing data serialization and deserialization processes, ensuring consistency in data handling across the system.
- The feature must adhere to the application's coding standards and best practices, ensuring maintainability and ease of future enhancements.

Save Command

Description

The 'Save Command' feature in the 'Litsa Minecraft Cobol' application provides administrators with the ability to save the current state of the game world, ensuring that progress is not lost during gameplay or server operations. This feature is implemented as a server-side command ('/save') that integrates seamlessly with the server management system. When executed, the command triggers the server's save functionality, which serializes and persists the game state, including player data, world chunks, and other critical information. The save process is designed to be efficient and minimally disruptive, ensuring that gameplay is not significantly impacted. Additionally, the feature includes error handling to notify administrators of any issues during the save process, such as file system errors or insufficient permissions. This functionality is essential for maintaining data integrity and providing a reliable gaming experience for players.

Acceptance Criteria

- The '/save' command must be registered and accessible to users with the appropriate permissions (e.g., administrators or server operators).
- Executing the '/save' command must trigger the server's save functionality, which serializes and persists the current game state, including player data, world chunks, and other relevant information.
- The save process must be efficient, ensuring minimal disruption to ongoing gameplay and server performance.
- The feature must include robust error handling to detect and report issues during the save process, such as file system errors, insufficient permissions, or disk space limitations.
- Administrators must receive a confirmation message upon successful execution of the '/save' command, indicating that the game state has been saved.
- In the event of an error, the feature must provide a clear and descriptive error message to the administrator, detailing the nature of the issue and potential steps to resolve it.
- The save functionality must integrate with the server's logging system, recording the timestamp and status of each save operation for auditing and debugging purposes.
- The feature must support both manual execution of the '/save' command and integration with automated save mechanisms (e.g., periodic autosaving).
- The save functionality must be compatible with the server's existing data serialization and storage systems, ensuring consistency and reliability.
- The feature must be tested across different server configurations and operating systems to ensure compatibility and stability.

Time Command

Description

The 'Time Command' feature in the 'Litsa Minecraft Cobol' server allows players or administrators to adjust the in-game time, providing control over the game environment. This feature is implemented as the '/time' command, which is registered within the server's command system. Users can set the time to predefined values such as 'day', 'night', 'noon', or 'midnight', or specify custom time values in ticks. The command integrates seamlessly with the world management system, ensuring that the in-game time is updated in real-time and synchronized across all connected clients. This feature is particularly useful for creating specific gameplay scenarios, testing, or managing the server environment during events. The implementation includes robust error handling to validate user input and provide meaningful feedback in case of invalid commands.

Acceptance Criteria

- The '/time' command must be registered and accessible to users with the appropriate permissions (e.g., administrators or players with operator privileges).
- The command must support predefined time values such as 'day', 'night', 'noon', and 'midnight', allowing users to quickly set the in-game time.
- The command must also allow users to specify custom time values in ticks (e.g., '/time set 1000').
- The in-game time must update immediately upon executing the command and be synchronized across all connected clients.
- The feature must include error handling to validate user input. For example, if an invalid time value or syntax is provided, the server should return an appropriate error message without crashing.
- The command must support additional subcommands such as '/time add ' to increment the current time by a specified number of ticks.
- The feature must integrate with the world management system to ensure that time changes are reflected in all relevant game mechanics, such as mob spawning and crop growth.
- The command must log its usage in the server console for administrative tracking and debugging purposes.
- The feature must demonstrate high performance, ensuring that time adjustments do not introduce noticeable delays or lag for players.
- The implementation must be compatible with the latest version of the server and tested across different configurations to ensure consistent behavior.
- The feature must adhere to the server's modular design principles, allowing for future extensions or modifications without impacting other systems.

Collision Detection for Block Placement

Description

The 'Collision Detection for Block Placement' feature in the 'Litsa Minecraft Cobol' app ensures that players cannot place blocks in spaces that are already occupied by other blocks, entities, or players. This feature integrates seamlessly with the item placement logic to validate the placement position before allowing the block to be placed. By checking for collisions during the placement process, it prevents unrealistic or erroneous block placements, maintaining the integrity of the game world and enhancing the overall gameplay experience. The system uses Axis-Aligned Bounding Box (AABB) collision detection to determine if the target space is free, ensuring accurate and efficient validation. This feature is critical for creating a realistic and error-free environment, especially in multiplayer scenarios where multiple players interact with the same game world.

Acceptance Criteria

- The system must check for collisions using Axis-Aligned Bounding Box (AABB) collision detection before allowing a block to be placed.
- Blocks cannot be placed in spaces already occupied by other blocks, entities, or players.
- The feature must integrate with the item placement logic to validate placement positions in real-time.
- The collision detection system must handle edge cases, such as partial overlaps or blocks with custom properties (e.g., slabs, stairs).
- The feature must provide feedback to the player when a block placement is invalid, such as a visual indicator or an error message.
- The collision detection must work seamlessly in both single-player and multiplayer modes, ensuring consistent behavior across all scenarios.
- The system must account for dynamic changes in the game world, such as moving entities or blocks being destroyed, and revalidate placement positions accordingly.
- The feature must demonstrate high performance, ensuring that collision checks do not introduce noticeable delays during gameplay.
- The implementation must be compatible with the latest version of the app and tested across different devices and operating systems for consistency.
- The collision detection logic must be modular and extensible, allowing for future enhancements or integration with additional gameplay mechanics.

Custom Block Properties Configuration

Description

The 'Custom Block Properties Configuration' feature in the 'Litsa Minecraft Cobol' server allows blocks to have specific properties dynamically assigned during placement. This feature enhances gameplay depth and customization by enabling blocks to exhibit behaviors such as orientation, waterlogging, and other contextual attributes. The configuration process is driven by player input and environmental factors, ensuring that block properties are contextually appropriate. For example, a block's orientation may depend on the player's facing direction, while waterlogging may be determined by the presence of adjacent water sources. This feature integrates seamlessly with item placement logic and neighbor interaction, ensuring that block properties are updated in real-time and reflected accurately in the game world. It is designed to provide a robust and flexible system for managing block behaviors, improving both the player experience and the server's customization capabilities.

Acceptance Criteria

- Blocks must dynamically configure their properties (e.g., orientation, waterlogging) during placement based on player input and environmental factors.
- The feature must support a wide range of block properties, including but not limited to orientation, waterlogging, and custom metadata.
- Block orientation must be determined by the player's facing direction at the time of placement, ensuring intuitive and predictable behavior.
- Waterlogging must be automatically applied to blocks placed adjacent to water sources, with proper validation to ensure compatibility with the block type.
- The feature must integrate seamlessly with item placement logic, ensuring that block properties are correctly applied when items are used to place blocks.
- Neighbor interaction must be supported, allowing blocks to update their properties based on the state of adjacent blocks (e.g., connecting fences or walls).
- The system must provide real-time updates to clients, ensuring that block properties are accurately reflected in the game world immediately after placement.
- Error handling must be implemented to manage invalid configurations or unsupported block types, providing clear feedback to the player or server administrator.
- The feature must be compatible with existing block and item logic, ensuring no conflicts or regressions in functionality.
- Performance must be optimized to handle dynamic property configuration without introducing significant delays or server lag, even in high-load scenarios.
- The feature must be thoroughly tested across different Minecraft versions supported by the server to ensure compatibility and stability.
- Documentation must be provided for developers and server administrators, detailing how to extend or customize block properties for specific use cases.

Modular Design for Item and Block Logic

Description

The 'Modular Design for Item and Block Logic' feature in the 'Litsa Minecraft Cobol' app introduces a streamlined and reusable approach to implementing behaviors for items and blocks. By leveraging modular components and callbacks, this feature allows developers to define specific logic for items and blocks in a consistent and maintainable manner. The modular design integrates seamlessly with the item registration and placement workflows, enabling efficient customization and scalability. This approach reduces code duplication, simplifies debugging, and ensures that similar behaviors across different items and blocks can be implemented with minimal effort. The feature is particularly beneficial for managing complex interactions, such as custom block properties, item-specific actions, and dynamic updates, while maintaining a clean and organized codebase.

Acceptance Criteria

- The modular design must provide a reusable framework for defining item-specific and block-specific logic, reducing code duplication across the codebase.
- Developers must be able to register new items and blocks with their associated logic using a consistent and intuitive API.
- The feature must support the use of callbacks to handle custom behaviors, such as block placement, item usage, and interaction events.
- The modular components must integrate seamlessly with the existing item registration and placement workflows, ensuring compatibility with other features of the app.
- The system must allow for the easy addition of new behaviors or modification of existing ones without requiring significant changes to the core codebase.
- The feature must include comprehensive documentation and examples to help developers understand and utilize the modular design effectively.
- The implementation must demonstrate high performance, ensuring that the modular design does not introduce significant overhead during runtime.
- The feature must be thoroughly tested to ensure that all registered items and blocks behave as expected, with no unintended side effects.
- Error handling must be implemented to provide clear feedback to developers in case of invalid configurations or logic definitions.
- The modular design must support backward compatibility with existing items and blocks, ensuring that previously implemented logic continues to function correctly.
- The system must be scalable, allowing for the addition of a large number of items and blocks without degrading performance or maintainability.
- The feature must adhere to coding standards and best practices, ensuring that the modular design is clean, readable, and maintainable.

Resource Pack Availability Notification

Description

The 'Resource Pack Availability Notification' feature in the 'Litsa Minecraft Cobol' server ensures that clients are informed about the availability of resource packs. This feature is designed to enhance the user experience by enabling players to customize their gameplay visuals and sounds. When a client connects to the server, the server sends a packet containing details about available resource packs, including their namespace, ID, and version. This information allows players to select and apply resource packs that are compatible with the server. The feature integrates seamlessly with the resource management system, ensuring that clients always have access to the latest resource packs. Additionally, it supports versioning to prevent compatibility issues and ensures that resource packs are delivered efficiently and securely.

Acceptance Criteria

- The server must send a packet to the client upon connection, containing details about all available resource packs, including their namespace, ID, and version.
- The resource pack notification must integrate with the resource management system to ensure that the list of resource packs is up-to-date and accurate.
- The client must be able to parse the resource pack details and display them in a user-friendly interface, allowing players to view and select resource packs.
- The feature must support versioning to ensure that clients only receive resource packs compatible with their game version.
- The server must handle cases where no resource packs are available by sending an empty notification or a default message to the client.
- The notification process must be optimized for performance, ensuring minimal impact on server and client resources during the transmission of resource pack details.
- The feature must include error handling to manage scenarios such as missing resource pack files, corrupted data, or network interruptions, providing appropriate feedback to the client.
- The resource pack notification must comply with security standards, ensuring that no unauthorized or malicious resource packs are included in the list.
- The feature must be tested across different client versions and platforms to ensure compatibility and consistent behavior.

- The server must log resource pack notifications for debugging and auditing purposes, including details about the resource packs sent to each client.

Symmetry and Mirroring for Recipes

Description

The 'Symmetry and Mirroring for Recipes' feature in the 'Litsa Minecraft Cobol' app enhances the crafting system by supporting both symmetrical and asymmetrical crafting recipes. This feature ensures that recipes are accurately represented and usable within the game, regardless of their symmetry. The system implements logic to handle symmetrical patterns seamlessly and introduces mirroring functionality for horizontally asymmetrical patterns. This allows players to input recipes in a mirrored format without errors, improving usability and reducing crafting frustrations. The feature is tightly integrated with the recipe parsing and management systems, ensuring that all recipes, whether symmetrical or asymmetrical, are processed and validated correctly. This functionality is essential for maintaining consistency and flexibility in the crafting experience, especially for complex or custom recipes.

Acceptance Criteria

- The crafting system must support symmetrical recipes, ensuring that identical patterns produce the same output regardless of orientation.
- Horizontally asymmetrical recipes must be processed correctly, with support for mirrored input patterns that yield the same output.
- The system must validate recipes during parsing, ensuring that both symmetrical and asymmetrical recipes are accurately represented in the game.
- The mirroring logic must be implemented in a way that does not introduce performance bottlenecks, even with a large number of recipes.
- The feature must integrate seamlessly with the existing recipe parsing and management systems, ensuring no conflicts or errors during recipe registration.
- Players must be able to input mirrored patterns in the crafting grid, and the system should recognize and process them as valid recipes.
- The crafting interface must provide clear feedback to players when a mirrored or symmetrical recipe is successfully crafted.
- The feature must be tested across various crafting scenarios, including custom recipes, to ensure consistent behavior and accuracy.
- Error handling must be implemented to provide meaningful feedback if an invalid recipe is detected during parsing or crafting.
- The feature must be compatible with the latest version of the app and tested across different devices and operating systems for consistency.
- The system must log recipe parsing and mirroring operations for debugging and analytics purposes, ensuring traceability of any issues.
- The feature must adhere to the app's modular design principles, allowing future updates or extensions to the crafting system without significant rework.

Entity and Player Removal

Description

The 'Entity and Player Removal' feature in the 'Litsa Minecraft Cobol' server ensures the game world remains accurate and optimized by removing entities or players that are no longer active. This feature is critical for maintaining the integrity of the game state and ensuring a seamless experience for active players. It integrates with the client-side rendering and player management systems to notify clients about the removal of specific entities or players. The server identifies inactive entities or disconnected players, updates the game state accordingly, and sends the necessary packets to clients to reflect these changes in real-time. This feature also ensures that resources associated with removed entities or players are released, optimizing server performance and memory usage.

Acceptance Criteria

- The server must accurately identify entities or players that are no longer active, such as disconnected players or entities that have been removed from the game world.
- Upon identifying an inactive entity or player, the server must send a removal notification packet to all connected clients, ensuring the game state is synchronized across all players.
- Clients must correctly process the removal notification and update their rendering and player lists to reflect the changes in the game world.
- The feature must ensure that resources associated with removed entities or players, such as memory and processing power, are released promptly to optimize server performance.
- The removal process must handle edge cases, such as entities being removed while players are interacting with them, without causing errors or crashes.
- The feature must include robust error handling to manage potential issues, such as network latency or packet loss, ensuring the game state remains consistent.
- The removal of players must trigger appropriate server-side events, such as saving player data and updating the player count in the server status.
- The feature must be compatible with the latest version of the server and tested across different client platforms to ensure consistent behavior.

- The server logs must include detailed information about entity and player removal events for debugging and auditing purposes.
- The feature must adhere to security best practices, ensuring that only authorized entities or players are removed and that the removal process cannot be exploited by malicious actors.

Chunk Center Update

Description

The 'Chunk Center Update' feature in the 'Litsa Minecraft Cobol' application is designed to optimize rendering and data management by dynamically updating the center of the chunk cache. This feature ensures that the server efficiently calculates and communicates the updated chunk center coordinates to the client, enabling smooth rendering and reducing unnecessary data processing. By keeping the chunk cache aligned with the player's position, the system minimizes memory usage and improves performance, especially in scenarios involving large-scale world exploration. This feature is critical for maintaining a seamless gameplay experience and ensuring that the server-client synchronization remains efficient and responsive.

Acceptance Criteria

- The server must calculate the updated chunk center coordinates based on the player's current position in the game world.
- The updated chunk center coordinates must be sent to the client in real-time, ensuring minimal delay in rendering updates.
- The client must use the received chunk center coordinates to adjust its rendering and data handling processes, ensuring efficient memory usage and smooth gameplay.
- The feature must handle edge cases, such as rapid player movement or teleportation, by recalculating and updating the chunk center without causing noticeable delays or performance degradation.
- The chunk center update process must be optimized to avoid unnecessary recalculations when the player remains within the same chunk.
- The feature must include error handling to manage potential issues, such as network latency or data corruption, ensuring the system remains stable and provides fallback mechanisms.
- The implementation must be compatible with the existing chunk management system, including loading, saving, and unloading of chunks.
- Performance metrics, such as the time taken to calculate and transmit chunk center updates, must be monitored and logged for debugging and optimization purposes.
- The feature must be tested across different devices and configurations to ensure consistent performance and compatibility.
- The chunk center update process must adhere to the application's modular design principles, allowing for future enhancements or integration with other features.

Held Item Updates

Description

The 'Held Item Updates' feature in the 'Litsa Minecraft Cobol' server ensures that the currently selected item in a player's hotbar is accurately synchronized between the server and the client. This feature is critical for maintaining consistency in gameplay, as it ensures that the item a player selects is correctly reflected in their inventory and user interface. When a player changes their held item, the server sends an update packet to the client, which then updates the UI to display the new item. This integration with the inventory and UI systems ensures a seamless experience for players, reducing discrepancies and improving gameplay accuracy. Additionally, the feature supports edge cases such as empty slots, item stacking, and quick item swaps, ensuring robust functionality.

Acceptance Criteria

- The server must detect when a player changes their held item in the hotbar and trigger an update event.
- The server must send a packet to the client containing the updated held item information, including item type, metadata, and quantity.
- The client must update the UI to reflect the new held item in the hotbar immediately upon receiving the update packet.
- The feature must handle edge cases, such as when the selected slot is empty, when the item is part of a stack, or when the player swaps items quickly.
- The held item updates must be synchronized with the inventory system, ensuring that the server's representation of the player's inventory matches the client's view.
- The feature must demonstrate high performance, with updates occurring in real-time without noticeable delays.
- The system must include error handling to manage potential issues, such as invalid item data or network packet loss, ensuring stability and providing fallback mechanisms.
- The feature must be compatible with all supported Minecraft versions and tested across different client configurations to ensure consistent behavior.
- The implementation must adhere to the modular design principles of the server, allowing for easy maintenance and future enhancements.
- The feature must log held item update events for debugging and analytics purposes, providing insights into player behavior and system performance.

Networking

Socket Communication

Description

The 'Socket Communication' feature in the 'Litsa Minecraft Cobol' application provides robust support for reading and writing data over sockets, enabling reliable data exchange in networked environments. This feature is designed to handle both small and large data transfers efficiently, using chunked operations to manage large payloads. It integrates seamlessly with the application's socket management system, ensuring end-to-end communication capabilities. Developers can utilize this feature to establish and maintain stable connections, send and receive data packets, and handle network communication errors gracefully. The implementation includes methods for synchronous and asynchronous data operations, ensuring flexibility for various use cases. Additionally, the feature is optimized for performance, ensuring low latency and high throughput in data transmission.

Acceptance Criteria

- The feature must support both reading and writing data over sockets, with methods for synchronous and asynchronous operations.
- Chunked operations must be implemented to handle large data transfers, ensuring that data is split into manageable chunks and reassembled correctly on the receiving end.
- The socket communication system must integrate seamlessly with the application's existing socket management infrastructure, providing end-to-end communication capabilities.
- The feature must include error handling mechanisms to manage network interruptions, timeouts, and invalid data, ensuring the application remains stable during communication failures.
- Data transmission must demonstrate high performance, with low latency and high throughput, even under heavy network load.
- The feature must support configurable timeouts for socket operations, allowing developers to define acceptable wait times for data transmission and reception.
- The implementation must include logging for socket operations, providing detailed information about data transfers, errors, and connection states for debugging and monitoring purposes.
- The feature must be compatible with the application's cross-platform deployment strategy, ensuring consistent behavior across different operating systems and environments.
- The socket communication system must adhere to security best practices, including encryption support for sensitive data and protection against common network vulnerabilities such as man-in-the-middle attacks.
- The feature must be tested for scalability, ensuring it can handle multiple simultaneous connections without performance degradation.
- The implementation must include comprehensive documentation and examples for developers, detailing how to use the feature for various network communication scenarios.

Ping Request Handling

Description

The 'Ping Request Handling' feature in the 'Litsa Minecraft Cobol' server ensures robust network diagnostics and stability by responding to client ping requests. When a client sends a ping request, the server decodes the request, processes the payload, and sends a response with the same payload to measure latency. After completing the response, the server terminates the connection with the client. This feature is part of the server's network management system and is designed to provide accurate latency measurements while maintaining server performance and security. It is particularly useful for debugging network issues and ensuring the server's responsiveness to client requests.

Acceptance Criteria

- The server must correctly decode incoming ping requests from clients, ensuring the payload is accurately interpreted.
- The server must send a response to the client with the exact same payload as received in the ping request, ensuring consistency and accuracy in latency measurement.
- The server must terminate the connection with the client immediately after sending the ping response, ensuring no lingering connections that could impact server performance.
- The feature must handle high volumes of ping requests without significant degradation in server performance, ensuring scalability and reliability.
- The server must log each ping request and response, including the latency measurement, for diagnostic and monitoring purposes.
- The feature must include error handling to manage malformed ping requests, ensuring the server remains stable and does not crash or behave unpredictably.
- The feature must comply with the server's security protocols, ensuring that no unauthorized or malicious ping requests can exploit the system.
- The implementation must be compatible with the server's existing network management system and adhere to the defined modular architecture.
- The feature must be tested across different network conditions (e.g., high latency, packet loss) to ensure consistent and reliable behavior.
- The feature must support cross-platform deployment, ensuring it functions correctly on all supported operating systems and environments.

Server Status Request Handling

Description

The 'Server Status Request Handling' feature in the 'Litsa Minecraft Cobol' application provides clients with real-time server status information. This includes the server's Message of the Day (MOTD), maximum player capacity, and current player count. The feature is designed to help players make informed decisions about joining the server by presenting them with essential server details before connecting. It works by aggregating server data, such as the number of active players and server properties, and sending a status packet to the client upon request. This feature integrates seamlessly with the server's configuration files and client communication systems, ensuring accurate and up-to-date information is delivered efficiently. It also supports customization of the MOTD and other server properties, allowing server administrators to personalize the server's presentation to potential players.

Acceptance Criteria

- The server must respond to status requests from clients with a status packet containing the MOTD, maximum player capacity, and current player count.
- The MOTD must be customizable through the server's configuration files and should support formatting options such as colors and special characters.
- The feature must accurately aggregate and display the current number of active players on the server at the time of the request.
- The maximum player capacity must be retrieved from the server's configuration and included in the status response.
- The status packet must be sent in a format that adheres to the Minecraft protocol specifications, ensuring compatibility with client expectations.
- The feature must handle multiple simultaneous status requests efficiently without impacting server performance.
- In cases where the server is offline or unable to process the request, a default or error response must be sent to the client, indicating the server's unavailability.
- The feature must log all status requests for monitoring and debugging purposes, including timestamps and client details.
- The implementation must include error handling to manage potential issues, such as invalid configuration values or network interruptions, ensuring the server remains stable.
- The feature must be tested across different Minecraft client versions to ensure compatibility and consistent behavior.
- The server status information must be updated dynamically, reflecting real-time changes such as player joins or leaves without requiring a server restart.
- The feature must support localization, allowing the MOTD and other status messages to be displayed in different languages based on client preferences.

Utility

Bit-Level Operations

Description

The 'Bit-Level Operations' feature in the 'Litsa Minecraft Cobol' application provides essential low-level data processing capabilities by performing precise bit-level calculations. This feature is designed to calculate the number of leading zero bits in a 32-bit integer using built-in functions, ensuring high performance and accuracy. It is particularly useful in scenarios requiring bit-level precision, such as data compression algorithms, cryptographic operations, and other optimization tasks. By leveraging this feature, developers can efficiently handle bitwise operations without the need for external libraries, ensuring seamless integration into the application's core logic. The implementation is optimized for performance and adheres to best practices for low-level data manipulation, making it a reliable tool for advanced computational tasks.

Acceptance Criteria

- The feature must accurately calculate the number of leading zero bits in a 32-bit integer using built-in functions, ensuring correctness for all possible input values.
- The implementation must demonstrate high performance, with minimal computational overhead, making it suitable for use in real-time or performance-critical applications.
- The feature must be thoroughly tested with a comprehensive set of test cases, including edge cases such as integers with all bits set to 0 or 1.
- The functionality must be accessible through a well-documented API or utility function, allowing developers to easily integrate it into their workflows.
- The feature must support cross-platform compatibility, ensuring consistent behavior across different operating systems and environments.
- Error handling must be implemented to manage invalid inputs or unexpected scenarios, providing meaningful feedback to developers.
- The implementation must include inline comments and documentation explaining the logic and purpose of the bit-level calculations, aiding future developers in understanding and maintaining the code.
- The feature must be compatible with the latest version of the application and should not introduce any regressions or conflicts with existing functionality.
- The feature must be validated for use in scenarios requiring bit-level precision, such as compression algorithms or cryptographic operations, ensuring its reliability in these contexts.
- The feature must adhere to the application's coding standards and guidelines, ensuring consistency with the rest of the codebase.

Infrastructure & Deployment

Version Compatibility and Update Process

Description

The 'Version Compatibility and Update Process' feature in the 'Litsa Minecraft Cobol' application ensures that the server remains compatible with the latest Minecraft versions, maintaining relevance and user satisfaction. This feature provides a structured and automated workflow for updating server code, data, and configurations to align with new Minecraft releases. It includes mechanisms for detecting version changes, validating compatibility, and applying updates without disrupting existing server functionality. The process is designed to minimize downtime and ensure a seamless transition to newer versions, while also preserving backward compatibility where feasible. Additionally, it incorporates robust error handling and rollback capabilities to address potential issues during the update process. This feature is critical for maintaining a high-quality user experience and ensuring the server's long-term viability in a rapidly evolving ecosystem.

Acceptance Criteria

- The server must detect and notify administrators of new Minecraft version releases, providing detailed information about the changes and their potential impact on the server.
- The update process must include a pre-update validation step to ensure that the server's current configuration, plugins, and data are compatible with the new version.
- The server must support automated updates for core components, including code, data structures, and configuration files, while allowing manual intervention if needed.
- Backward compatibility must be maintained for at least one previous Minecraft version, ensuring that users on older versions can still connect and play without issues.
- The update process must include a rollback mechanism to revert to the previous version in case of critical errors or compatibility issues during the update.
- The server must log all update-related activities, including detected version changes, applied updates, and any errors encountered, for auditing and troubleshooting purposes.
- The update process must be designed to minimize downtime, with a target of less than 5 minutes for typical updates under normal conditions.
- The server must provide a test environment where administrators and developers can simulate updates and verify compatibility before applying them to the live server.
- The update process must include automated testing to validate the functionality of critical server features (e.g., player connections, world generation, and inventory management) after the update is applied.
- The server must notify connected players of impending updates, providing a countdown and clear instructions to ensure a smooth transition.
- The update process must include mechanisms for synchronizing changes across distributed server instances in multi-server deployments, ensuring consistency and reliability.
- The feature must be compatible with the server's cross-platform deployment capabilities, ensuring that updates can be applied seamlessly across all supported platforms.
- The update process must adhere to security best practices, ensuring that no sensitive data is exposed or compromised during the update.
- The feature must be thoroughly documented, including step-by-step instructions for administrators and developers, as well as troubleshooting guides for common issues.

Cross-Platform Build System

Description

The 'Cross-Platform Build System' feature in the 'Litsa Minecraft Cobol' application ensures seamless server compilation and execution across diverse environments, catering to the needs of developers and administrators. This system leverages GnuCOBOL, C++, and a Makefile to compile the server into a single binary, enabling both native and Docker-based deployments. By abstracting platform-specific complexities, it provides a consistent and reliable build process, ensuring compatibility across operating systems and containerized environments. The feature is designed to streamline the development workflow, reduce setup time, and enhance deployment flexibility, making it easier to maintain and scale the server infrastructure.

Acceptance Criteria

- The build system must support compilation of the server into a single binary using GnuCOBOL, C++, and a Makefile, ensuring compatibility with both native and Docker-based environments.
- The system must provide clear and detailed documentation for setting up the build process, including prerequisites, commands, and troubleshooting steps.
- The Makefile must include platform-specific targets (e.g., Linux, Windows, macOS) to ensure compatibility across different operating systems.
- The Docker-based deployment must utilize a multi-stage build process to optimize the size and performance of the final container image.

- The build process must validate dependencies and provide meaningful error messages if any required tools or libraries are missing.
- The system must support version-specific builds, allowing developers to compile the server for specific Minecraft versions as needed.
- The build process must be automated and reproducible, ensuring consistent results across different environments and team members.
- The compiled binary must pass all integration tests to verify its functionality and compatibility with the intended deployment environment.
- The system must include a mechanism to clean up intermediate files and artifacts generated during the build process, ensuring efficient use of disk space.
- The Docker image must include all necessary runtime dependencies and configurations to run the server without additional setup.
- The build system must be tested and verified on at least three major operating systems (e.g., Linux, Windows, macOS) to ensure cross-platform compatibility.
- The feature must include a CI/CD pipeline integration to automate the build and deployment process, ensuring rapid and reliable updates.
- The system must provide logging and debugging options during the build process to assist developers in identifying and resolving issues.
- The final binary and Docker image must meet performance benchmarks, ensuring that the server runs efficiently in production environments.

Infrastructure Management

Server Initialization

Description

The 'Server Initialization' feature in the 'Litsa Minecraft Cobol' application is responsible for starting the server and generating a default configuration file (`server.properties`). This feature ensures that the server environment is properly set up and ready for operation. Upon execution of the 'Server' program, the system creates the `server.properties` file, which contains key server settings such as player limits, game mode, and network configurations. Administrators can edit this file to customize the server's behavior according to their requirements. The feature includes robust error handling to validate the configuration file and provide meaningful feedback in case of invalid or missing values. This functionality is essential for enabling administrators to effectively manage and maintain the server environment.

Acceptance Criteria

- The server must successfully initialize when the 'Server' program is executed, creating a default `server.properties` file in the designated directory.
- The `server.properties` file must include all essential server settings, such as player limits, game mode, difficulty, and network configurations, with default values pre-populated.
- The feature must validate the generated configuration file to ensure all required fields are present and contain valid values. If any issues are detected, the system must provide clear error messages and guidance for resolution.
- The server initialization process must log key events, such as the creation of the configuration file and any errors encountered, to a log file for debugging and auditing purposes.
- The feature must support re-initialization without overwriting existing `server.properties` files unless explicitly instructed by the administrator.
- The server must remain stable and operational after initialization, with all default settings applied correctly.
- The feature must be compatible with cross-platform deployment environments, including Docker containers, ensuring consistent behavior across different operating systems.
- The initialization process must complete within a reasonable time frame (e.g., under 5 seconds) to ensure a smooth user experience.
- The feature must include comprehensive documentation and comments in the codebase to assist developers in understanding and maintaining the initialization logic.
- The `server.properties` file must be human-readable and formatted for easy editing by administrators.
- The feature must include unit tests to validate the creation and validation of the `server.properties` file, ensuring reliability and preventing regressions.

Infrastructure

Modular Design Architecture

Description

The 'Modular Design Architecture' feature in the 'Litsa Minecraft Cobol' application organizes the system into distinct, self-contained modules, each responsible for specific tasks. This modular approach promotes reusability, maintainability, and scalability by isolating functionalities such as file handling, JSON processing, and recipe loading into independent components. Developers can easily update or replace individual modules without affecting the rest of the system, simplifying the integration of new features and reducing the risk of introducing bugs. This architecture also enhances collaboration by allowing teams to work on separate modules concurrently, improving development efficiency. The modular design ensures that the application remains robust and adaptable to future requirements.

Acceptance Criteria

- The system must be divided into clearly defined modules, each responsible for a specific functionality, such as file handling, JSON processing, and recipe loading.
- Modules must be self-contained, with minimal dependencies on other modules, ensuring that changes in one module do not impact others.
- The architecture must support the addition of new modules without requiring significant changes to the existing codebase.
- Each module must have a well-documented interface, including input and output specifications, to facilitate integration and usage by other parts of the system.
- The modular design must allow for independent testing of each module, ensuring that bugs can be isolated and resolved efficiently.
- The system must demonstrate improved maintainability by enabling developers to update or replace individual modules without affecting the overall application functionality.
- The architecture must support concurrent development by multiple teams, allowing them to work on separate modules without conflicts.
- Modules must be reusable across different parts of the application, reducing code duplication and improving consistency.
- The system must include a mechanism for managing module dependencies, ensuring that all required modules are loaded and initialized correctly during runtime.
- The modular design must be compatible with the existing build and deployment processes, including support for cross-platform deployment using Docker and GnuCOBOL.
- The architecture must be scalable, allowing the application to handle increased complexity or additional features without significant refactoring.
- The modular design must be documented in the project's technical documentation, including guidelines for creating, updating, and integrating modules.

Server Operations

Server Initialization

Description

The 'Server Initialization' feature in the 'Litsa Minecraft Cobol' application serves as the foundational entry point for starting the server. It ensures that all critical components, such as region file management, registry management, and server properties configuration, are properly initialized and validated before the server becomes operational. This feature integrates seamlessly with other server components to prepare the environment for gameplay. It handles loading configuration files, validating settings, and initializing essential resources like regions and registries. Additionally, it includes robust error handling to detect and report invalid or missing configurations, ensuring a smooth startup process. This feature is designed to provide a reliable and efficient server initialization workflow, enabling developers and administrators to focus on higher-level tasks without worrying about the underlying setup complexities.

Acceptance Criteria

- The server must successfully load all required configuration files during initialization, including server properties, region files, and registry data.
- The feature must validate the integrity and correctness of configuration files, providing clear error messages for invalid or missing values.
- Region file management must be initialized, ensuring that all necessary region data is loaded and ready for use.
- Registry management must be properly set up, including the initialization of block, item, and entity registries.
- The server must log detailed information about the initialization process, including timestamps and status updates for each step.
- The initialization process must handle errors gracefully, preventing the server from crashing and providing actionable feedback to the user.
- The server must be ready to accept client connections only after all initialization steps are successfully completed.
- The feature must support cross-platform deployment, ensuring compatibility with different operating systems and environments.
- The initialization process must demonstrate high performance, completing within a reasonable time frame even for large configurations.
- The feature must include unit and integration tests to verify the correctness of the initialization workflow.
- The server must provide a clear notification (e.g., console message or log entry) indicating that initialization is complete and the server is ready for operation.
- The feature must be compatible with the latest version of the application and tested across supported Minecraft versions.
- The initialization process must support modular configuration, allowing administrators to enable or disable specific features or components as needed.

Communication and Control

Packet and Command Handling

Description

The 'Packet and Command Handling' feature in the 'Litsa Minecraft Cobol' application is a critical component that facilitates seamless communication between the server and connected clients. It ensures that the server can process incoming packets from clients and respond appropriately, while also providing a robust command system for administrative control. The server registers handlers for various packet types based on the current state of the client (e.g., handshake, login, play) and processes these packets to update the game state or provide feedback to the client. Additionally, the feature includes a command registration system that allows administrators to execute server commands such as 'gamemode', 'save', and 'stop' for managing gameplay and server operations. This feature is designed to be extensible, allowing developers to add new packet types or commands as needed. It ensures high performance, error handling, and compatibility with the server's modular architecture, making it a cornerstone of the server's communication and control capabilities.

Acceptance Criteria

- The server must register handlers for all supported packet types, categorized by client states (e.g., handshake, login, play).
- Incoming packets must be validated for correctness and security before processing to prevent malicious activity or data corruption.
- The server must process packets efficiently, ensuring minimal latency in communication between clients and the server.
- The command system must support a predefined set of administrative commands, including but not limited to 'gamemode', 'save', and 'stop'.
- Commands must be executed with appropriate permissions, ensuring that only authorized users (e.g., server administrators) can perform administrative tasks.
- The feature must include error handling for invalid packets or commands, providing meaningful feedback to the client or administrator.
- The server must log all executed commands and significant packet events for auditing and debugging purposes.
- The packet handling system must be extensible, allowing developers to add new packet types or modify existing ones without disrupting the server's operation.
- The command system must support autocompletion for registered commands, improving usability for administrators.
- The feature must demonstrate high performance, with packet processing and command execution occurring within acceptable timeframes to ensure a smooth user experience.
- The system must handle edge cases, such as malformed packets or unsupported commands, gracefully without crashing the server.
- The feature must be compatible with the server's modular design, ensuring seamless integration with other components such as player management, world updates, and inventory handling.
- The feature must be tested across different Minecraft versions supported by the server to ensure compatibility and reliability.
- The packet and command handling system must adhere to security best practices, ensuring that sensitive data is not exposed and that unauthorized access is prevented.

Security & Compliance

Boundary Checking

Description

The 'Boundary Checking' feature in the 'Litsa Minecraft Cobol' application ensures that all positions within the game world adhere to predefined vertical boundaries. This feature is critical for maintaining game rules and preventing unintended behavior, such as players or entities moving outside the valid world space. It integrates seamlessly with the block manipulation and entity management systems, validating positions during operations like block placement, entity movement, and collision detection. By enforcing these constraints, the feature ensures a consistent and stable gameplay experience while safeguarding the integrity of the game world. Additionally, it provides error handling and feedback mechanisms to notify users or systems when an operation violates the boundary rules.

Acceptance Criteria

- The system must define and enforce vertical boundaries for the game world, with configurable minimum and maximum Y-coordinates.
- All position-based operations, including block placement, entity movement, and collision detection, must validate positions against the defined boundaries.
- If a position is found to be outside the valid boundaries, the system must prevent the operation and provide appropriate feedback, such as an error message or log entry.
- Boundary checking must integrate with the block manipulation system to ensure that blocks cannot be placed or modified outside the valid vertical range.
- Boundary checking must integrate with the entity management system to ensure that entities cannot move or spawn outside the valid vertical range.
- The feature must handle edge cases, such as positions at the exact boundary limits, without causing errors or inconsistencies.
- Boundary validation must be performed efficiently to avoid impacting the performance of the game loop or other critical systems.
- The feature must include unit tests and integration tests to verify its functionality across various scenarios, including normal operations and edge cases.
- Boundary checking must be compatible with the latest version of the server and tested across different configurations to ensure consistent behavior.
- The system must log boundary violations for debugging and monitoring purposes, including details such as the position, operation type, and entity or block involved.
- The feature must support future updates to the world boundaries, allowing administrators to modify the vertical limits without requiring significant code changes.

Compression and Decompression Utilities

Description

The 'Compression and Decompression Utilities' feature in the 'Litsa Minecraft Cobol' application provides robust functionality for compressing and decompressing data using Zlib and Gzip formats. This feature is designed to optimize storage and data transfer efficiency by reducing the size of data files without compromising their integrity. It seamlessly integrates with other modules, such as encoding and decoding utilities, to ensure that data is stored and transmitted in a compact format. By leveraging these utilities, the application minimizes resource usage, enhances performance, and supports efficient data handling across various workflows. This feature is essential for maintaining a scalable and resource-efficient server environment, particularly in scenarios involving large datasets or frequent data exchanges.

Acceptance Criteria

- The feature must support both Zlib and Gzip compression formats, ensuring compatibility with industry-standard data handling practices.
- Compression and decompression operations must be efficient, with minimal impact on server performance, even when processing large datasets.
- The utilities must integrate seamlessly with other modules, such as NBT encoding/decoding and JSON parsing, to enable compact data storage and transmission.
- Compressed data must retain its integrity and be fully recoverable upon decompression, ensuring no data loss or corruption.
- The feature must include error handling to manage scenarios such as invalid input data, insufficient memory, or decompression failures, providing clear feedback to the user or system logs.
- The utilities must support both synchronous and asynchronous workflows, allowing flexibility in how they are used within the application.
- Compression and decompression operations must be tested across various data types and sizes to ensure consistent performance and reliability.
- The feature must include logging capabilities to track compression and decompression events, including details such as data size before and after compression and any errors encountered.
- The utilities must adhere to security best practices, ensuring that compressed data does not expose vulnerabilities such as buffer overflows or unauthorized access.

- The feature must be compatible with the latest version of the application and tested across different deployment environments, including containerized setups using Docker.
- Documentation must be provided for developers, detailing how to use the compression and decompression utilities, including examples and integration guidelines.

Stop Command

Description

The 'Stop Command' feature in the 'Litsa Minecraft Cobol' server provides administrators with a reliable and controlled way to shut down the server. This feature registers the '/stop' command, which, when executed, triggers the server's stop functionality. The stop process ensures that all active game states are saved, client connections are gracefully terminated, and resources are released. This integration with the server management system guarantees a smooth shutdown process, preventing data loss and maintaining server integrity. The feature is designed to be robust, handling edge cases such as ongoing player actions or unsaved data, and provides feedback to the administrator about the shutdown progress.

Acceptance Criteria

- The '/stop' command must be registered and accessible only to users with administrator privileges.
- Executing the '/stop' command must initiate a controlled shutdown process, ensuring all active game states are saved to disk.
- The server must notify all connected clients about the impending shutdown, providing a customizable message and sufficient time for players to disconnect.
- All client connections must be gracefully terminated during the shutdown process, ensuring no abrupt disconnections.
- The server must release all allocated resources, including memory, file handles, and network sockets, to prevent resource leaks.
- The shutdown process must log detailed information about the steps being performed, including saving game states, disconnecting clients, and releasing resources.
- The feature must handle edge cases, such as ongoing player actions or unsaved data, by ensuring all critical operations are completed before the server stops.
- The server must provide feedback to the administrator during the shutdown process, including progress updates and confirmation of a successful shutdown.
- The '/stop' command must be tested across different server configurations and environments to ensure consistent behavior.
- Error handling must be implemented to manage potential issues during the shutdown process, such as file write failures or network interruptions, with appropriate fallback mechanisms.
- The feature must integrate seamlessly with other server management functionalities, such as autosaving and logging, to ensure a cohesive shutdown process.
- The '/stop' command must adhere to security best practices, ensuring that unauthorized users cannot execute the command.

Error Handling and Validation

Description

The 'Error Handling and Validation' feature in the 'Litsa Minecraft Cobol' application ensures robust data integrity and system stability by validating input data and gracefully managing errors. This feature is designed to handle scenarios such as invalid JSON structures, unknown tags, and missing or corrupted data. It integrates seamlessly with all data processing modules, ensuring that errors are detected early and managed effectively. When an error is encountered, the system either recovers by applying fallback mechanisms or terminates the affected process in a controlled manner, preventing disruptions to gameplay or server operations. Additionally, detailed error logs are generated to assist developers and administrators in diagnosing and resolving issues. This feature is critical for maintaining a reliable and secure environment for both players and server operators.

Acceptance Criteria

- The system must validate all incoming data, including JSON structures, tags, and required fields, before processing.
- Invalid JSON inputs must trigger an error response, with detailed logs specifying the nature and location of the issue.
- Unknown tags or unsupported data formats must be flagged, and the system should either ignore them or apply a predefined fallback mechanism.
- Missing critical data must result in a controlled termination of the affected process, with appropriate error messages displayed to the user or logged for administrators.
- The system must handle corrupted data gracefully, ensuring that gameplay or server operations are not disrupted.
- Error logs must include timestamps, error codes, and detailed descriptions to facilitate debugging and resolution by developers or administrators.
- The feature must integrate with all data processing modules, ensuring consistent error handling across the application.
- The system must provide feedback to users or administrators when errors occur, including suggestions for corrective actions where applicable.
- Error handling must be tested across various scenarios, including edge cases, to ensure comprehensive coverage and reliability.
- The feature must demonstrate high performance, ensuring that error validation and handling do not introduce significant delays in data processing.
- The system must include safeguards to prevent cascading failures caused by unhandled errors in interconnected modules.

- The error handling mechanism must comply with security best practices, ensuring that sensitive information is not exposed in error messages or logs.
- The feature must be compatible with the latest version of the application and tested across different deployment environments, including containerized setups.
- The system must support configurable error thresholds, allowing administrators to define acceptable levels of error tolerance for specific operations.

Keep-Alive Mechanism

Description

The 'Keep-Alive Mechanism' feature in the 'Litsa Minecraft Cobol' app ensures a stable and uninterrupted connection between the client and server during gameplay. This feature periodically sends keep-alive packets from the server to connected clients to verify the connection status. If a client fails to respond within a specified timeout period, the server identifies the connection as inactive and disconnects the client to free up resources. This mechanism is critical for maintaining a seamless multiplayer experience by promptly detecting and handling connection issues. It also helps optimize server performance by ensuring only active connections are maintained.

Acceptance Criteria

- The server must send periodic keep-alive packets to all connected clients at configurable intervals.
- Clients must respond to keep-alive packets within a specified timeout period to confirm an active connection.
- If a client fails to respond within the timeout period, the server must disconnect the client and log the event for debugging purposes.
- The keep-alive interval and timeout period must be configurable through the server's configuration files.
- The feature must handle edge cases, such as network latency, by allowing a reasonable buffer time before disconnecting a client.
- The mechanism must be lightweight and efficient, ensuring minimal impact on server performance and bandwidth usage.
- The server must provide detailed logs for keep-alive events, including timestamps, client identifiers, and disconnection reasons, to assist in debugging and monitoring.
- The feature must be compatible with all supported Minecraft versions and tested across different client platforms to ensure consistent behavior.
- The keep-alive mechanism must integrate seamlessly with the server's connection handling system, ensuring no conflicts with other features like handshake and login management.
- The feature must include error handling to manage scenarios such as packet loss or malformed keep-alive responses, ensuring the server remains stable.
- The implementation must adhere to security best practices, ensuring that keep-alive packets cannot be exploited for malicious purposes, such as denial-of-service attacks.

Error Handling and Validation

Description

The 'Error Handling and Validation' feature in the 'Litsa Minecraft Cobol' server ensures robust data consistency and security by validating all client actions and interactions with the server. This feature is designed to detect and handle invalid or malicious behavior from clients, such as sending malformed packets, attempting unauthorized actions, or exceeding defined boundaries. When such behavior is identified, the server takes appropriate actions, including logging the incident, notifying administrators, and disconnecting the offending client to maintain the integrity of the game environment. The feature integrates seamlessly with all client-server communication systems, ensuring that validation checks are performed at every stage of interaction. Additionally, it provides detailed error messages and logs to aid in debugging and monitoring. This feature is critical for maintaining a secure, stable, and fair multiplayer experience.

Acceptance Criteria

- The server must validate all incoming client packets for correctness, including structure, data types, and expected values.
- Invalid or malformed packets must trigger appropriate error handling routines, including logging the error and disconnecting the client if necessary.
- The server must validate client actions, such as block placement, movement, and inventory updates, ensuring they adhere to game rules and boundaries.
- Any unauthorized or malicious actions, such as attempting to access restricted commands or exceeding defined boundaries, must result in immediate disconnection of the client and logging of the incident.
- Error logs must include detailed information, such as the client UUID, IP address, timestamp, and the nature of the error, to assist in debugging and monitoring.
- The feature must provide clear and user-friendly error messages to clients when they are disconnected due to validation failures, explaining the reason for the disconnection.
- The error handling routines must be designed to ensure the server remains stable and operational, even when handling a high volume of invalid requests.
- Validation mechanisms must be integrated with all client-server communication systems, including packet handling, command execution, and gameplay interactions.
- The feature must include safeguards to prevent false positives, ensuring legitimate client actions are not incorrectly flagged as invalid.

- The server must support configurable thresholds for certain validation checks, such as packet rate limits, to allow customization based on server capacity and requirements.
- The feature must be tested across various scenarios, including edge cases and stress tests, to ensure reliability and robustness.
- Error handling routines must include mechanisms to notify server administrators of critical incidents, such as repeated malicious attempts from a specific client or IP address.
- The feature must comply with data privacy and security standards, ensuring that sensitive client information is not exposed in error logs or messages.

Performance Optimization

View Distance Optimization

Description

The 'View Distance Optimization' feature in the 'Litsa Minecraft Cobol' server ensures efficient memory and computational resource usage by dynamically unloading chunks that fall outside the player's configured view distance. This feature integrates seamlessly with the chunk management system to identify and unload unnecessary chunks while maintaining the integrity of the game world. It also notifies clients of changes to ensure a smooth gameplay experience. By reducing the active chunk load, this feature significantly improves server performance, especially in multiplayer environments with multiple players exploring different areas of the world. The implementation is designed to be non-intrusive, ensuring that gameplay remains uninterrupted while optimizing server resources.

Acceptance Criteria

- The server must dynamically identify chunks that fall outside the player's configured view distance and unload them to free up memory and computational resources.
- Chunks within the player's view distance must remain loaded and accessible, ensuring seamless gameplay without visible delays or interruptions.
- The feature must integrate with the chunk management system to handle chunk loading and unloading efficiently, avoiding redundant operations.
- Clients must be notified of chunk changes (e.g., unloading or loading) to ensure their game state remains synchronized with the server.
- The unloading process must not disrupt gameplay, ensuring that players do not experience lag or stuttering when moving through the world.
- The feature must support configurable view distance settings, allowing server administrators to adjust the view distance based on server capacity and player needs.
- The optimization must handle edge cases, such as players rapidly moving between areas, by preloading chunks near the view distance boundary to prevent delays.
- The system must demonstrate measurable performance improvements, such as reduced memory usage and lower CPU load, when compared to scenarios without view distance optimization.
- The feature must be compatible with multiplayer environments, ensuring that each player's view distance is managed independently without affecting other players.
- The implementation must include error handling to manage scenarios where chunk unloading or client notifications fail, ensuring server stability.
- The feature must be tested across different server configurations and player counts to ensure consistent performance and reliability.
- The optimization must not interfere with other server features, such as spawn area management or persistent chunk loading for specific regions.

Developer Tools

Utility Functions for World Operations

Description

The 'Utility Functions for World Operations' feature in the 'Litsa Minecraft Cobol' app provides a set of reusable helper functions designed to streamline common world-related tasks. These functions include capabilities such as dropping items at specific locations, retrieving the world's age, and other frequently used operations. By integrating seamlessly with systems like entity management and world metadata, these utility functions enhance the development process and gameplay features. They are designed to be modular, efficient, and easy to use, enabling developers to focus on higher-level functionality while ensuring consistency and reliability in world operations. This feature is essential for simplifying complex workflows and improving the maintainability of the codebase.

Acceptance Criteria

- The utility functions must include a method for dropping items at specified world coordinates, ensuring proper integration with the entity management system.
- A function to retrieve the world's age must be implemented, providing accurate and up-to-date information from the world metadata.
- All utility functions must be modular and reusable, allowing developers to easily integrate them into various parts of the application without duplication of code.
- The functions must demonstrate high performance, with minimal impact on server tick times, even when called frequently during gameplay or server operations.
- Error handling must be implemented for all utility functions, ensuring stability in cases of invalid inputs or unexpected conditions (e.g., invalid coordinates or missing metadata).
- The utility functions must be thoroughly documented, including usage examples and parameter descriptions, to assist developers in understanding and utilizing them effectively.
- Unit tests must be provided for each utility function, covering a wide range of scenarios to ensure reliability and correctness.
- The functions must be compatible with the latest version of the server and tested across different configurations to ensure consistent behavior.
- Integration with existing systems, such as entity management and world metadata, must be seamless, with no conflicts or redundant functionality.
- The utility functions must adhere to the coding standards and best practices established for the 'Litsa Minecraft Cobol' project, ensuring maintainability and readability.

Data Processing

Variable-Length Integer Encoding

Description

The 'Variable-Length Integer Encoding' feature in the 'Litsa Minecraft Cobol' application optimizes the storage and transmission of integer values by encoding them into variable-length formats. This approach reduces the data size for smaller values, improving overall efficiency in data handling. The feature is designed to handle a wide range of integer values, including positive, negative, and large numbers, ensuring accurate length calculation and encoding. It integrates seamlessly with the broader data serialization system, enhancing the application's ability to process and transmit data efficiently. This feature is particularly useful in scenarios where minimizing data size is critical, such as network communication and file storage. Developers can rely on this feature to ensure that integer data is encoded in a compact and efficient manner without compromising accuracy or compatibility.

Acceptance Criteria

- The feature must accurately encode positive, negative, and large integer values into variable-length formats, ensuring no data loss or corruption.
- The encoded output must be validated to ensure it adheres to the expected variable-length integer format, with correct length calculation for all input values.
- The feature must integrate seamlessly with the existing data serialization system, allowing encoded integers to be used in broader workflows without additional modifications.
- Performance benchmarks must demonstrate that the encoding process is efficient, with minimal overhead compared to traditional fixed-length encoding methods.
- The feature must include comprehensive test coverage, verifying correct encoding and decoding for a wide range of integer values, including edge cases such as the smallest and largest representable integers.
- Error handling must be implemented to manage invalid input values or unexpected scenarios, providing clear feedback to developers.
- The feature must support compatibility with other components of the application that rely on encoded integer data, ensuring smooth interoperability.
- Documentation must be provided, explaining the encoding algorithm, its integration points, and usage examples for developers.
- The feature must be tested across different environments and platforms to ensure consistent behavior and compatibility.
- The implementation must adhere to coding standards and best practices, ensuring maintainability and readability for future development.

3D Position Encoding

Description

The '3D Position Encoding' feature in the 'Litsa Minecraft Cobol' application provides a robust mechanism for encoding 3D spatial coordinates (X, Y, Z) into binary formats. This feature ensures precise representation of spatial data, which is critical for applications like mapping, gaming, and simulation. It validates the encoding process to guarantee accuracy for both zero and non-zero values, ensuring that the encoded data is reliable and consistent. The feature is designed to support systems that depend on precise spatial data, such as dynamic terrain generation, collision detection, and entity positioning. By leveraging efficient binary encoding, it optimizes data storage and transmission, making it suitable for high-performance applications. This feature is implemented with a focus on accuracy, performance, and compatibility with existing systems.

Acceptance Criteria

- The feature must accurately encode 3D coordinates (X, Y, Z) into binary formats, ensuring no loss of precision for both zero and non-zero values.
- The encoding process must support a wide range of coordinate values, including negative, positive, and zero values, without errors or inconsistencies.
- The encoded binary data must be compatible with other features and systems within the application, such as chunk management, entity positioning, and collision detection.
- The feature must include validation mechanisms to detect and handle invalid input coordinates, providing meaningful error messages or logs for debugging.
- The encoding process must demonstrate high performance, with minimal latency, even when processing large datasets or high-frequency updates.
- The feature must be thoroughly tested across different scenarios, including edge cases like maximum and minimum coordinate values, to ensure reliability.
- The implementation must adhere to the application's existing data serialization standards, ensuring seamless integration with other components.
- The feature must include comprehensive documentation for developers, detailing the encoding algorithm, input/output formats, and integration points.

- The encoded data must be optimized for storage and transmission, minimizing the size of the binary representation without compromising accuracy.
- The feature must be compatible with the application's cross-platform deployment requirements, ensuring consistent behavior across different environments.

JSON Parsing Placeholder

Description

The 'JSON Parsing Placeholder' feature in the 'Litsa Minecraft Cobol' application represents the foundational implementation for future JSON parsing capabilities. This feature is designed to complement the existing JSON encoding functionality, enabling the application to handle incoming JSON data effectively. By establishing a framework for bidirectional data exchange, this feature ensures that the application is prepared to process structured JSON inputs, paving the way for seamless integration with external systems and APIs. The placeholder includes basic scaffolding and validation mechanisms to ensure compatibility with the application's architecture, while also adhering to best practices for data integrity and error handling. This feature is a critical step toward expanding the application's data processing capabilities and enhancing its interoperability with modern data formats.

Acceptance Criteria

- The application must include a placeholder module for JSON parsing, with clearly defined interfaces and methods for future implementation.
- The placeholder must be integrated into the application's existing data processing pipeline, ensuring compatibility with the JSON encoding feature.
- The feature must include basic validation mechanisms to ensure that incoming JSON data adheres to the expected structure and format.
- The placeholder must provide detailed logging for any attempted JSON parsing operations, aiding in debugging and future development.
- The feature must include comprehensive documentation outlining its purpose, workflow, and integration points within the application.
- The placeholder must be designed to handle potential errors gracefully, such as malformed JSON inputs, without causing application crashes or instability.
- The feature must be tested across different environments to ensure consistent behavior and compatibility with the application's architecture.
- The implementation must adhere to the application's coding standards and best practices, ensuring maintainability and scalability for future enhancements.

End Compound Handling

Description

The 'End Compound Handling' feature in the 'Litsa Minecraft Cobol' application ensures robust and accurate processing of the end of compound structures within NBT (Named Binary Tag) data. This feature is critical for maintaining data integrity during decoding operations. It identifies the end of compound structures in NBT data streams and ensures that no parsing errors occur due to improperly terminated or malformed data. By implementing this feature, the system can reliably decode and process complex data structures, which is essential for the seamless operation of the game server. Additionally, this feature includes comprehensive testing workflows to validate its functionality, ensuring that edge cases and malformed data are handled gracefully without compromising the stability of the application. This feature is particularly useful for developers working on data serialization and Quality Assurance Engineers testing the robustness of the system.

Acceptance Criteria

- The system must correctly identify the end of compound structures in NBT data streams, ensuring no parsing errors occur.
- Malformed or improperly terminated compound structures must be detected, and appropriate error messages should be logged without causing application crashes.
- The feature must maintain data integrity during decoding, ensuring that all valid data is processed accurately and no data is lost or corrupted.
- Comprehensive unit tests must be implemented to validate the feature's functionality, covering edge cases such as nested compound structures and empty compounds.
- Performance benchmarks must demonstrate that the feature does not introduce significant overhead during data parsing operations.
- The feature must be compatible with existing NBT encoding and decoding workflows, ensuring seamless integration with other data serialization features.
- Error handling mechanisms must be in place to gracefully manage unexpected scenarios, such as corrupted data streams or unsupported data formats.
- The implementation must adhere to the application's coding standards and be thoroughly documented to assist future developers in understanding and maintaining the feature.
- The feature must be tested across different environments and configurations to ensure consistent behavior and reliability.
- Logs generated during the handling of compound structures must provide sufficient detail for debugging and monitoring purposes without overwhelming the system with excessive output.

Skip Functionality

Description

The 'Skip Functionality' feature in the 'Litsa Minecraft Cobol' application enhances the efficiency of the NBT (Named Binary Tag) decoder by enabling it to bypass specific tag types during data traversal. This functionality is particularly useful when processing large datasets or irrelevant data segments, as it reduces unnecessary overhead and improves performance. The feature supports skipping over various tag types, including arrays, strings, lists, and nested compounds, ensuring seamless navigation through complex data structures. By integrating directly with the NBT decoder, it validates the ability to handle diverse tag types while maintaining data integrity and optimizing traversal speed. This feature is essential for scenarios where selective data processing is required, such as when only specific tags are relevant to the operation being performed.

Acceptance Criteria

- The decoder must successfully skip over specified tag types, including arrays, strings, lists, and nested compounds, without processing their contents.
- The feature must validate the ability to handle diverse tag types, ensuring no data corruption or unintended side effects during the skipping process.
- The skipping mechanism must integrate seamlessly with the NBT decoder, maintaining compatibility with existing decoding workflows.
- Performance benchmarks must demonstrate a measurable improvement in data traversal speed when skipping irrelevant or large data segments.
- The feature must include robust error handling to manage scenarios where invalid or unsupported tag types are encountered, ensuring the decoder remains stable.
- The implementation must be thoroughly tested with a variety of NBT data structures, including edge cases such as deeply nested compounds and empty tags.
- The feature must provide clear logging or debugging information to indicate which tags were skipped during the decoding process, aiding in troubleshooting and validation.
- The skipping functionality must be configurable, allowing developers to specify which tag types should be skipped based on the application's requirements.
- The feature must adhere to the application's existing coding standards and be compatible with the latest version of the NBT decoder.
- Documentation must be provided to explain the usage, configuration, and limitations of the skip functionality, ensuring ease of adoption by developers.

Testing & Validation

Data Type Parsing Validation

Description

The 'Data Type Parsing Validation' feature in the 'Litsa Minecraft Cobol' application ensures robust and reliable parsing of various JSON data types, including null, boolean, integer, and float values. This feature is designed to validate the application's ability to handle diverse data types, ensuring comprehensive support for JSON structures. It includes rigorous testing of the JSON parsing module to handle edge cases such as invalid formats, missing values, and unexpected data types. By implementing this feature, the application guarantees accurate data processing and prevents potential errors caused by malformed or unsupported JSON inputs. This feature is critical for maintaining data integrity and ensuring seamless interaction with external data sources.

Acceptance Criteria

- The JSON parsing module must correctly identify and process null, boolean, integer, and float values from JSON inputs without errors.
- Edge cases, such as invalid formats (e.g., malformed JSON, incorrect data types) and missing values, must be handled gracefully, with appropriate error messages or fallback mechanisms.
- The feature must include unit tests that cover all supported data types, including edge cases, to ensure comprehensive validation.
- The application must log detailed error messages for invalid JSON inputs, enabling developers to debug and resolve issues efficiently.
- The parsing module must demonstrate high performance, processing large JSON datasets without significant delays or memory overhead.
- The feature must ensure compatibility with the latest JSON standards and maintain backward compatibility with older JSON structures used in the application.
- The validation process must not interfere with other application workflows, ensuring seamless integration with existing features.
- The feature must be tested across different environments and platforms to ensure consistent behavior and reliability.
- The implementation must adhere to the application's security standards, ensuring that no sensitive data is exposed or mishandled during the parsing process.
- The feature must include documentation for developers, detailing the supported data types, edge cases, and error handling mechanisms.

Value Skipping Functionality

Description

The 'Value Skipping Functionality' feature in the 'Litsa Minecraft Cobol' application enhances the JSON parsing module by allowing it to efficiently skip over irrelevant JSON values during parsing. This functionality is designed to optimize performance and resource usage, especially when processing large JSON payloads where only specific parts of the data are required. The feature supports skipping over various JSON value types, including numbers, strings, objects, and arrays, ensuring flexibility and robustness. By focusing on relevant data, this feature reduces memory overhead and parsing time, making it particularly useful for scenarios involving selective data extraction. Additionally, the feature includes comprehensive testing to validate its ability to handle diverse JSON structures and edge cases, ensuring reliability and stability in production environments.

Acceptance Criteria

- The JSON parsing module must successfully skip over irrelevant JSON values, including numbers, strings, objects, and arrays, without affecting the integrity of the remaining data.
- The feature must demonstrate improved performance by reducing parsing time and memory usage when processing large JSON payloads with irrelevant data.
- The functionality must be compatible with nested JSON structures, ensuring that irrelevant values within deeply nested objects or arrays are correctly skipped.
- The feature must include robust error handling to manage malformed JSON inputs, ensuring the application remains stable and provides meaningful error messages when necessary.
- Comprehensive unit tests must be implemented to validate the feature's ability to skip over all supported JSON value types, including edge cases such as empty objects, arrays, and null values.
- The feature must integrate seamlessly with existing JSON parsing workflows, ensuring no disruption to other modules or features that rely on the parsing module.
- Performance benchmarks must demonstrate measurable improvements in resource usage and parsing speed when the feature is enabled, compared to scenarios where all JSON values are processed.
- The feature must be documented thoroughly, including usage guidelines, supported JSON value types, and examples of expected behavior, to assist developers in understanding and utilizing the functionality.
- The implementation must adhere to coding standards and best practices, ensuring maintainability and ease of future enhancements.
- The feature must be tested across different environments and configurations to ensure consistent behavior and compatibility with the application's deployment scenarios.

Data Serialization

NBT Data Encoding and Validation

Description

The 'NBT Data Encoding and Validation' feature in the 'Litsa Minecraft Cobol' application ensures the accurate encoding and validation of various data types into the NBT (Named Binary Tag) format. This format is essential for storing and transmitting structured data within the Minecraft-like environment. The feature supports a wide range of data types, including byte, int, long, float, double, string, arrays, lists, compounds, and UUIDs. It handles edge cases such as empty arrays, unnamed data, and invalid data structures, ensuring robust data integrity. This feature is tightly integrated with the broader data serialization and deserialization processes, enabling seamless data handling across the application. By validating and encoding data correctly, it guarantees compatibility with the NBT format and prevents data corruption or loss during storage or transmission.

Acceptance Criteria

- The feature must support encoding and validation for all standard NBT data types, including byte, int, long, float, double, string, arrays, lists, compounds, and UUIDs.
- Edge cases, such as empty arrays, unnamed data, and invalid data structures, must be handled gracefully without causing application crashes or data corruption.
- The encoded NBT data must strictly adhere to the NBT format specifications, ensuring compatibility with other systems or tools that use the format.
- The feature must integrate seamlessly with the application's data serialization and deserialization workflows, ensuring consistent data handling across all modules.
- Validation errors must provide clear and actionable error messages, enabling developers to identify and resolve issues quickly.
- The feature must include unit tests covering all supported data types and edge cases, ensuring reliability and correctness.
- Performance benchmarks must demonstrate that the encoding and validation processes operate efficiently, even with large or complex data structures.
- The feature must support backward compatibility with previously encoded NBT data, ensuring that older data can still be read and validated correctly.
- The implementation must include comprehensive documentation for developers, detailing the supported data types, edge cases, and integration points with other modules.
- The feature must be tested across different environments and configurations to ensure consistent behavior and compatibility.

File Management

Region File Naming

Description

The 'Region File Naming' feature in the 'Litsa Minecraft Cobol' application ensures standardized and consistent naming conventions for region data files. This feature calculates file names based on the X and Z coordinates of the region, supporting both positive and negative values. The naming convention follows a structured format, such as 'r.X.Z.mca', where X and Z represent the region's coordinates. By adhering to this format, the feature facilitates efficient organization, retrieval, and management of region data within the file storage system. It seamlessly integrates with the application's file handling mechanisms, ensuring compatibility with other features like chunk management and world storage. This feature is critical for maintaining a scalable and organized file system, especially in large, dynamically generated worlds.

Acceptance Criteria

- The feature must generate file names in the format 'r.X.Z.mca', where X and Z are the region's coordinates.
- The naming convention must support both positive and negative coordinate values, ensuring compatibility with all regions in the world.
- File names must be unique for each region, preventing conflicts or overwrites in the file storage system.
- The feature must integrate seamlessly with the file storage system, ensuring that region files are stored and retrieved correctly based on their calculated names.
- The file naming logic must be consistent across all parts of the application, including chunk management, world storage, and region file handling.
- The feature must handle edge cases, such as extremely large or small coordinate values, without errors or performance degradation.
- The implementation must include error handling to manage invalid inputs or failures in file operations, providing meaningful feedback to the user or developer.
- The feature must be tested across different operating systems and file systems to ensure compatibility and consistent behavior.
- The naming convention must align with existing Minecraft region file standards to ensure interoperability with external tools and systems.
- The feature must demonstrate high performance, even when generating file names for a large number of regions in quick succession.

Utilities

String Manipulation Utilities

Description

The 'String Manipulation Utilities' feature in the 'Litsa Minecraft Cobol' application provides a set of robust utility functions for handling and processing strings, specifically designed to streamline text-related operations. This feature focuses on trimming file extensions from strings, ensuring accurate and efficient processing of file names. It handles various scenarios, including strings with no extensions, multiple extensions, or invalid inputs, making it a versatile tool for file management and user input validation. By offering a consistent and reliable way to manipulate strings, this feature simplifies workflows for developers and system administrators, reducing the likelihood of errors and improving overall application stability.

Acceptance Criteria

- The utility must correctly identify and remove a single file extension from a string, leaving the base file name intact.
- The utility must handle cases where no file extension is present, returning the original string without modification.
- The utility must process strings with multiple extensions (e.g., 'file.tar.gz') by removing only the last extension, leaving the rest intact.
- The utility must validate inputs and handle invalid cases (e.g., null, empty strings, or non-string inputs) gracefully, returning appropriate error messages or default values.
- The utility must support trimming extensions for both standard file names (e.g., 'example.txt') and unconventional formats (e.g., '.hiddenfile').
- The utility must be optimized for performance, ensuring quick processing even for large datasets or batch operations.
- The utility must be thoroughly tested with a variety of edge cases, including special characters, long file names, and unusual extensions.
- The feature must include comprehensive documentation, including usage examples and guidelines for integration into other parts of the application.
- The utility must be compatible with the application's existing file management and user input validation systems, ensuring seamless integration.
- The feature must adhere to coding standards and best practices, ensuring maintainability and readability for future developers.

UUID Conversion

Description

The 'UUID Conversion' feature in the 'Litsa Minecraft Cobol' application provides a robust mechanism for converting UUIDs (Universally Unique Identifiers) between binary and string representations. This feature ensures consistent formatting and validation of UUIDs across the application, supporting critical workflows such as data serialization, user identification, and cross-system compatibility. By enabling seamless conversion, it simplifies the handling of UUIDs in various contexts, including database storage, network communication, and user management. The implementation adheres to industry standards for UUID formatting, ensuring interoperability with external systems and APIs. This feature is essential for maintaining data integrity and ensuring smooth interactions between different components of the application.

Acceptance Criteria

- The feature must support conversion of UUIDs from binary format to string format, ensuring the output adheres to the standard UUID string representation (e.g., 'xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx').
- The feature must support conversion of UUIDs from string format to binary format, ensuring the binary output is compact and optimized for storage or transmission.
- The conversion process must validate the input format, rejecting invalid UUID strings or binary data with appropriate error messages or exceptions.
- The feature must handle edge cases, such as null or empty inputs, and provide meaningful feedback to the calling function or user.
- The implementation must ensure high performance, capable of processing large volumes of UUID conversions without significant delays.
- The feature must be compatible with the application's existing data serialization and deserialization workflows, ensuring seamless integration.
- The converted UUIDs must maintain consistency across different systems and platforms, ensuring interoperability with external APIs and services.
- The feature must include unit tests to validate the accuracy and reliability of the conversion process, covering a wide range of test cases.
- The feature must log errors or warnings in case of invalid inputs or unexpected failures, aiding in debugging and monitoring.
- The implementation must adhere to the application's coding standards and best practices, ensuring maintainability and readability of the codebase.
- The feature must be documented thoroughly, including usage examples and guidelines for developers integrating it into other parts of the application.

User Management

Kill Command

Description

The 'Kill Command' feature in the 'Litsa Minecraft Cobol' server allows administrators or players with operator permissions to execute the '/kill' command to eliminate a specified player. This command reduces the target player's health to zero, triggering the standard death sequence, including death animations and the broadcasting of a customizable death message to all players in the server. The feature is seamlessly integrated with the command system and player management modules, ensuring proper validation of user input and smooth execution of associated events. This feature enhances administrative control by providing a quick and efficient way to manage disruptive players or test gameplay scenarios, while also adding a dynamic element to gameplay interactions.

Acceptance Criteria

- The '/kill' command must be registered within the server's command system and accessible to users with the appropriate permissions (e.g., administrators or operators).
- The command must accept a valid target player name or UUID as input. If no target is specified, the command defaults to killing the player who issued it.
- Executing the command must immediately reduce the target player's health to zero, triggering the standard death sequence, including animations and the removal of the player from the active game state.
- A customizable death message must be broadcasted to all players in the server, indicating the target player's death and the cause (e.g., 'Player1 was killed by an operator').
- The feature must validate the input to ensure the specified target player exists and is currently online. If the target is invalid or offline, an appropriate error message must be displayed to the command issuer.
- The command must integrate with the player management system to ensure that all associated events, such as inventory drops and respawn handling, are triggered correctly.
- The feature must include error handling to manage edge cases, such as attempting to kill a non-existent player or executing the command during server downtime.
- The '/kill' command must be logged in the server's activity logs, including details such as the issuer, target, and timestamp, for administrative auditing purposes.
- The feature must demonstrate high performance, ensuring that the command execution and associated events occur without noticeable delays, even under high server load.
- The feature must be compatible with the latest version of the server and tested across different configurations to ensure consistent behavior.
- The command must adhere to the server's permission system, ensuring that only authorized users can execute it.
- The feature must support localization, allowing the death message to be displayed in the server's configured language.

Whitelist Management Command

Description

The 'Whitelist Management Command' feature in the 'Litsa Minecraft Cobol' server application provides server administrators with a robust set of tools to manage player access. This feature ensures controlled access to the server by enabling administrators to enable, disable, reload, list, add, or remove players from the whitelist. It integrates seamlessly with the server's command system, offering a user-friendly interface for executing whitelist-related operations. Each command is registered with detailed help text and callbacks for execution, ensuring clarity and ease of use. By leveraging this feature, administrators can enhance server security and maintain a well-managed player base. The feature is designed to handle edge cases, such as invalid player names or duplicate entries, with appropriate error messages and feedback to the user.

Acceptance Criteria

- The whitelist management commands must be accessible through the server's command system, with clear syntax and help text for each operation.
- Administrators must be able to enable or disable the whitelist functionality using commands, with immediate effect on server access.
- The feature must support adding players to the whitelist by username, ensuring that duplicate entries are not allowed and providing feedback for invalid usernames.
- Administrators must be able to remove players from the whitelist by username, with confirmation messages indicating successful removal.
- A command must be available to list all players currently on the whitelist, displaying their usernames in a readable format.
- The feature must include a reload command to refresh the whitelist from the server's configuration files, ensuring changes made outside the server are applied without requiring a restart.
- Error handling must be implemented for scenarios such as invalid commands, missing parameters, or network issues, with user-friendly error messages provided.
- The commands must be logged in the server's console for auditing purposes, including the administrator's username and the action performed.

- The whitelist management commands must be tested for performance and reliability, ensuring they execute quickly and do not impact server stability.
- The feature must adhere to the server's permission system, restricting access to whitelist commands to authorized server administrators only.
- The commands must be compatible with the server's modular design, ensuring they can be extended or modified without affecting other features.
- The feature must support internationalization, allowing help text and error messages to be displayed in multiple languages based on server configuration.

Player Addition Notification

Description

The 'Player Addition Notification' feature in the 'Litsa Minecraft Cobol' server ensures that clients are promptly informed whenever a new player joins the game. This feature is designed to enhance the multiplayer experience by keeping all connected clients updated about active players in real-time. When a new player connects to the server, the server sends a dedicated packet to all clients containing the player's details, such as UUID, username, and optional properties like game mode and display name. This packet is seamlessly integrated with the player management system, ensuring accurate and consistent updates. The feature also supports optional customization, such as displaying a formatted name or additional metadata, depending on the server's configuration. By providing this notification, the feature fosters better communication and interaction among players while maintaining synchronization between the server and clients.

Acceptance Criteria

- The server must send a 'Player Addition Notification' packet to all connected clients whenever a new player joins the game.
- The packet must include the player's UUID, username, and optional properties such as game mode and display name.
- The feature must integrate with the player management system to ensure accurate and up-to-date information about active players.
- Clients must correctly parse and display the received player details, ensuring consistency across all connected clients.
- The notification must be sent in real-time, with minimal delay, to maintain synchronization between the server and clients.
- The feature must handle edge cases, such as duplicate player entries or invalid data, by implementing robust error handling and validation mechanisms.
- The feature must support optional customization of player display names and metadata, based on server configuration settings.
- The notification system must be compatible with the latest version of the server and tested across different client implementations to ensure reliability.
- The feature must adhere to data security and privacy standards, ensuring that sensitive player information is not exposed or misused.
- The system must log player addition events for debugging and auditing purposes, providing detailed information about the notification process.
- The feature must demonstrate high performance, ensuring that the addition of new players does not introduce significant latency or impact server stability.
- The notification must be visually distinguishable in the client interface, ensuring players can easily identify new additions to the game.

Player Login Initialization

Description

The 'Player Login Initialization' feature in the 'Litsa Minecraft Cobol' server ensures a seamless and efficient start to a player's session. Upon login, the server sends essential game state information to the client, including the player's current dimension, game mode, and view distance settings. This process is designed to integrate smoothly with the client, initializing the game environment and ensuring that players are ready to interact with the world immediately after connecting. The feature also handles any necessary synchronization between the server and client to ensure consistency in the game state. This functionality is critical for maintaining a high-quality user experience and reducing potential delays or errors during the login process.

Acceptance Criteria

- The server must send the player's current dimension, game mode, and view distance settings to the client immediately upon login.
- The client must successfully receive and apply the dimension, game mode, and view distance information to initialize the game environment.
- The feature must ensure that the player's game state is synchronized between the server and client, avoiding discrepancies in the game environment.
- The login initialization process must handle edge cases, such as invalid or missing data, by providing default values or error messages to the client.
- The feature must demonstrate high performance, ensuring that the login process completes within a reasonable time frame (e.g., under 2 seconds) even under high server load.
- The server must validate the player's login credentials and protocol version before sending the initialization data to ensure compatibility and security.
- The feature must support multiple players logging in simultaneously without causing delays or conflicts in the initialization process.

- The server must log relevant events during the login initialization process, such as successful logins, errors, or mismatched data, for debugging and monitoring purposes.
- The feature must be compatible with the latest Minecraft version supported by the server and tested across different client configurations to ensure reliability.
- The login initialization process must include error handling for network interruptions or client disconnections, ensuring that the server remains stable and the player can retry the login process without issues.

Player Abilities Management

Description

The 'Player Abilities Management' feature in the 'Litsa Minecraft Cobol' server ensures that player abilities, such as flying, invulnerability, and creative mode, are dynamically updated based on the current game mode or server commands. This feature enhances gameplay flexibility by allowing seamless transitions between different player states. The server sends real-time updates to the client, ensuring that the player's abilities are accurately reflected in their game environment. For example, when a player switches to creative mode, they gain the ability to fly and become invulnerable, and these changes are immediately synchronized with the client. This feature is critical for maintaining consistency between the server and client states, providing a smooth and immersive gameplay experience.

Acceptance Criteria

- The server must correctly update player abilities (e.g., flying, invulnerability, creative mode) based on the player's current game mode or server commands.
- Player ability updates must be sent to the client in real-time, ensuring that the client reflects the player's current capabilities without delay.
- The feature must support all game modes (e.g., survival, creative, adventure, spectator) and correctly apply the corresponding abilities for each mode.
- The server must handle edge cases, such as transitioning from creative mode to survival mode, by disabling abilities like flying and ensuring the player takes fall damage if applicable.
- The feature must include error handling to manage invalid or unsupported game mode transitions, providing appropriate feedback to the server administrator or player.
- Player ability updates must be logged on the server for debugging and auditing purposes, including details such as the player ID, the updated abilities, and the triggering event (e.g., game mode change, command execution).
- The feature must be compatible with multiplayer environments, ensuring that ability updates for one player do not interfere with the abilities of other players.
- The client must receive visual and functional feedback for ability changes, such as enabling the flight toggle or updating the health bar for invulnerability.
- The feature must be tested across different devices and client versions to ensure consistent behavior and compatibility.
- The implementation must adhere to performance optimization standards, ensuring that ability updates do not introduce noticeable latency or server load.
- The feature must respect server configuration settings, such as disabling flying globally or restricting certain abilities to specific players or roles.
- The feature must integrate with the server's permission system, allowing administrators to grant or revoke specific abilities for individual players or groups.
- The feature must include support for custom game modes or plugins that modify player abilities, ensuring extensibility and compatibility with third-party modifications.

Player Respawn Handling

Description

The 'Player Respawn Handling' feature in the 'Litsa Minecraft Cobol' server ensures a seamless and consistent experience for players re-entering the game after death. When a player dies, the server automatically resets their health, position, and inventory state to predefined values or their last saved state. The server then sends a respawn packet to the client, which updates the player's view and reinitializes their gameplay environment. This feature is tightly integrated with the player state management system and world systems to ensure that respawn events are handled efficiently and without errors. Additionally, it supports custom respawn logic, such as respawning at a specific location (e.g., a bed or world spawn point) and resetting game mode or abilities if required. This feature is critical for maintaining gameplay continuity and providing a smooth return to the game world after a death event.

Acceptance Criteria

- The server must reset the player's health to the maximum value upon respawn.
- The player's position must be set to the appropriate respawn location, such as the world spawn point or a bed if available and valid.
- The player's inventory state must be reset according to the server's configuration, either clearing the inventory or restoring it to the last saved state.
- A respawn packet must be sent to the client, ensuring the player's view is updated to reflect the new position and state.
- The feature must integrate with the player state management system to ensure all player attributes (e.g., health, hunger, experience) are correctly initialized upon respawn.

- The feature must support custom respawn logic, such as game mode changes or ability resets, based on server settings or gameplay rules.
- The respawn process must be completed without noticeable delays, ensuring a smooth transition for the player.
- The feature must handle edge cases, such as invalid respawn locations, by falling back to a default spawn point.
- The server must log respawn events for debugging and analytics purposes, including the player's username, previous position, and respawn location.
- The feature must include error handling to manage potential issues, such as corrupted player data or network interruptions during the respawn process.
- The respawn handling must be compatible with multiplayer environments, ensuring that other players are notified of the respawn event if necessary (e.g., through chat or visual effects).
- The feature must be tested across different Minecraft versions supported by the server to ensure compatibility and consistent behavior.
- The respawn handling must adhere to the server's performance standards, ensuring minimal impact on tick rates or server responsiveness.

Client Configuration Management

Description

The 'Client Configuration Management' feature in the 'Litsa Minecraft Cobol' application processes client-specific settings to ensure a personalized and optimized gameplay experience. This feature handles client-provided configuration data, such as locale preferences, view distance, and skin customization. Upon receiving this data, the server validates and integrates it with existing server settings, ensuring compatibility and consistency. The server then sends back corresponding configuration packets to the client, enabling seamless synchronization of user preferences with the server environment. This feature is critical for enhancing user experience by tailoring gameplay settings to individual player needs while maintaining server performance and stability.

Acceptance Criteria

- The server must be able to receive and parse client configuration data, including but not limited to locale, view distance, and skin customization settings.
- Client configuration data must be validated to ensure it adheres to server-defined constraints and does not introduce inconsistencies or errors.
- The server must send appropriate configuration packets back to the client, reflecting the processed and validated settings.
- The feature must support dynamic updates to client settings during gameplay, allowing players to modify preferences without requiring a server restart.
- The integration of client settings with server configurations must ensure compatibility, avoiding conflicts that could impact gameplay or server performance.
- The feature must log configuration changes for debugging and auditing purposes, ensuring traceability of client-server interactions.
- The system must handle edge cases, such as missing or invalid configuration data, by applying default settings and notifying the client of the issue.
- The feature must demonstrate high performance, processing configuration data and sending responses with minimal latency to ensure a smooth user experience.
- The implementation must adhere to data security and privacy standards, ensuring that sensitive client information is not exposed or misused.
- The feature must be tested across different devices and client versions to ensure consistent behavior and compatibility.
- The user interface for client configuration must provide clear feedback to players, indicating whether their settings have been successfully applied or require adjustments.
- The feature must support localization, ensuring that configuration options and feedback messages are available in multiple languages as per the client's locale setting.
- The system must include error handling mechanisms to manage potential failures in packet transmission or processing, ensuring stability and providing meaningful feedback to the client.

Player Movement Processing

Description

The 'Player Movement Processing' feature in the 'Litsa Minecraft Cobol' server ensures accurate and fair gameplay by tracking and validating player movement updates. This feature processes position, rotation, and status updates received from the client, ensuring that all movements adhere to the game's rules and constraints. It integrates seamlessly with the teleportation acknowledgment system to prevent movement exploits, such as unauthorized teleportation or speed hacks. By maintaining a robust validation mechanism, this feature ensures that player actions are synchronized across the server and other clients, providing a consistent and cheat-free multiplayer experience. Additionally, it handles edge cases like boundary violations, collision detection, and latency-induced discrepancies, ensuring smooth and reliable gameplay for all users.

Acceptance Criteria

- The server must accurately process and validate player movement updates, including position, rotation, and status changes, ensuring they comply with game rules and constraints.
- The feature must integrate with the teleportation acknowledgment system to prevent movement exploits, such as unauthorized teleportation or speed hacks.
- Player movements must be synchronized across the server and all connected clients, ensuring consistency in the multiplayer environment.
- The system must handle edge cases, such as boundary violations, collision detection, and latency-induced discrepancies, without causing gameplay interruptions or errors.
- The feature must support real-time processing of movement updates, ensuring minimal latency and a smooth gameplay experience.
- In cases of invalid or suspicious movement data, the server must log the event and take appropriate action, such as rejecting the update or flagging the player for further review.
- The feature must be compatible with the latest version of the server and tested across different client configurations to ensure consistent behavior.
- The movement processing system must demonstrate high performance, capable of handling a large number of concurrent players without significant delays or resource bottlenecks.
- The feature must include error handling to manage potential issues, such as network interruptions or corrupted movement data, ensuring the server remains stable and responsive.
- The implementation must adhere to security best practices, ensuring that movement data cannot be tampered with or exploited by malicious clients.

Chat and Command Handling

Description

The 'Chat and Command Handling' feature in the 'Litsa Minecraft Cobol' server enables seamless communication and interaction within the game world. This feature processes player chat messages and commands, ensuring they are appropriately broadcasted or executed. Chat messages are formatted and sent to all connected players, fostering in-game communication. Commands entered by players are parsed, validated, and executed by the server, enabling various gameplay and administrative actions. The feature integrates with the system's chat messaging and command execution subsystems, ensuring a robust and efficient workflow. It also supports command autocompletion, providing players with a user-friendly interface for entering commands. This feature is essential for maintaining a dynamic and interactive multiplayer environment.

Acceptance Criteria

- The server must process and broadcast chat messages from players to all other connected players in real-time, ensuring minimal latency.
- Chat messages must include the sender's username and be formatted consistently to enhance readability.
- The server must parse and validate commands entered by players, ensuring they conform to the expected syntax and permissions.
- Commands must be executed accurately, with appropriate feedback provided to the player (e.g., success or error messages).
- The feature must support command autocompletion, providing players with suggestions as they type commands.
- Chat and command handling must integrate seamlessly with the server's logging system, recording all messages and commands for auditing and debugging purposes.
- The feature must include error handling to manage invalid commands or messages, providing clear feedback to the player without crashing the server.
- The system must support a configurable chat filter to block inappropriate or offensive language, ensuring a safe and respectful environment for players.
- The feature must handle high volumes of chat messages and commands without significant performance degradation, ensuring scalability for large multiplayer sessions.
- The chat and command handling system must respect player permissions, ensuring that only authorized users can execute administrative or restricted commands.
- The feature must support multilingual chat messages, allowing players to communicate in their preferred language without issues.
- The system must be compatible with the latest version of the server and tested across different platforms to ensure consistent behavior.
- The feature must adhere to accessibility standards, ensuring that players with disabilities can use the chat and command system effectively.

Teleportation Acknowledgment

Description

The 'Teleportation Acknowledgment' feature in the 'Litsa Minecraft Cobol' server ensures accurate player positioning during teleportation events by validating client responses to teleport IDs. When a teleportation event is triggered, the server assigns a unique teleport ID to the event and sends it to the client. The client must acknowledge the teleportation by responding with the corresponding teleport ID. The server then verifies the acknowledgment and updates the player's state accordingly. This process prevents desynchronization issues, such as players

appearing in incorrect locations or experiencing movement glitches. The feature is tightly integrated with the player movement processing system to ensure seamless transitions and accurate state management.

Acceptance Criteria

- The server must generate a unique teleport ID for each teleportation event and send it to the client as part of the teleportation packet.
- The client must respond with the correct teleport ID to acknowledge the teleportation event.
- The server must validate the client's response to ensure the teleport ID matches the one sent for the event.
- If the client fails to acknowledge the teleportation within a predefined timeout period, the server must log the event and take corrective action, such as resending the teleportation packet or notifying the player of a potential issue.
- The player's position and state must be updated on the server only after successful validation of the teleport ID acknowledgment.
- The feature must integrate with the player movement processing system to ensure smooth transitions and prevent movement-related desynchronization.
- The server must handle edge cases, such as network latency or packet loss, by implementing retry mechanisms and ensuring the teleportation process remains robust.
- The feature must log all teleportation events, including successful acknowledgments and failures, for debugging and monitoring purposes.
- The teleportation acknowledgment process must not introduce noticeable delays or performance issues, even under high server load or with multiple simultaneous teleportation events.
- The feature must be tested across different client versions and configurations to ensure compatibility and reliability.
- The implementation must adhere to security best practices, ensuring that teleportation events cannot be spoofed or manipulated by malicious clients.

Server State Management

Description

The 'Server State Management' feature in the 'Litsa Minecraft Cobol' application ensures a seamless and structured experience for clients during their connection lifecycle. This feature tracks and transitions client states through key phases, including handshake, login, and play. It integrates with the authentication system to validate client credentials and with the gameplay system to initialize the player's session. The server maintains a clear state machine to manage these transitions, ensuring that clients are always in a valid state. This feature is critical for maintaining server stability, preventing unauthorized access, and providing a smooth onboarding experience for players. Additionally, it supports error handling to gracefully manage invalid states or connection issues, ensuring the server remains robust and user-friendly.

Acceptance Criteria

- The server must track client states through the following phases: handshake, login, and play, ensuring that each client is in a valid state at all times.
- During the handshake phase, the server must validate the client's protocol version and prepare for the login process.
- The login phase must integrate with the authentication system to verify client credentials and generate a unique player UUID.
- The play phase must initialize the player's session, including loading player data (e.g., position, inventory, health) and transitioning the client to the active gameplay state.
- The server must handle invalid states gracefully, providing appropriate error messages to the client and logging the issue for debugging purposes.
- State transitions must be logged for auditing and debugging, including timestamps and client identifiers.
- The feature must support multiple concurrent clients, ensuring that state transitions are handled independently and do not interfere with one another.
- The server must disconnect clients that fail to complete the handshake or login phases within a configurable timeout period.
- The feature must integrate with the gameplay system to ensure that players are fully initialized before entering the play phase.
- The server must provide a mechanism to reset or recover client states in case of unexpected errors or disconnections.
- The implementation must be tested across different Minecraft versions supported by the server to ensure compatibility.
- The feature must include performance optimizations to handle high volumes of client connections without significant delays or resource bottlenecks.
- The server must provide detailed logs and metrics for state transitions, including the number of clients in each state and the average time spent in each phase.
- The feature must adhere to security best practices, ensuring that sensitive client data (e.g., authentication tokens) is not exposed during state transitions.

Utility Features

Replaceable Block Positioning

Description

The 'Replaceable Block Positioning' feature in the 'Litsa Minecraft Cobol' application ensures accurate block placement by determining whether the clicked block is replaceable and calculating the correct position for the new block. This feature is essential for maintaining seamless gameplay mechanics, as it supports scenarios where blocks like grass, snow layers, or other replaceable blocks need to be overridden by the newly placed block. The workflow involves checking the properties of the clicked block, identifying if it is replaceable, and then calculating the appropriate position for the new block. This feature integrates with other block placement functionalities, ensuring consistency and accuracy in block interactions.

Acceptance Criteria

- The system must correctly identify whether the clicked block is replaceable based on its properties (e.g., grass, snow layers, or other predefined replaceable blocks).
- If the clicked block is replaceable, the system must calculate the correct position for the new block to replace it seamlessly.
- If the clicked block is not replaceable, the system must calculate the position for the new block to be placed adjacent to the clicked block, following standard block placement rules.
- The feature must support all block types, including custom blocks with replaceable properties, ensuring compatibility with the game's block registry.
- The block placement logic must integrate with other block placement features, such as orientation, waterlogging, and custom block properties, ensuring consistent behavior.
- The system must provide visual feedback to the player, such as a placement preview, to indicate where the block will be placed before finalizing the action.
- The feature must handle edge cases, such as placing blocks at world boundaries or on non-solid surfaces, without causing errors or inconsistencies.
- The block placement process must be optimized for performance, ensuring minimal latency even in high-load scenarios with multiple players interacting simultaneously.
- The feature must include error handling to manage invalid block placements, such as attempting to place a block in an occupied or restricted position, and provide appropriate feedback to the player.
- The implementation must be thoroughly tested across different game modes (e.g., creative, survival) and scenarios to ensure consistent and reliable behavior.
- The feature must adhere to the game's modular design principles, allowing for easy updates or extensions to the block placement logic in the future.

Configuration Management

Feature Flags Synchronization

Description

The 'Feature Flags Synchronization' feature ensures that clients are consistently updated with the enabled feature flags configured on the server, such as 'minecraft:vanilla'. This synchronization is critical for maintaining compatibility and consistency between the server and client configurations. When the server initializes or updates its configuration, it sends a packet containing the active feature flags to all connected clients. This packet is processed by the client to align its behavior and settings with the server's active features. The feature integrates seamlessly with the server's configuration management system, ensuring that any changes to feature flags are automatically communicated to clients in real-time. This functionality is essential for preventing mismatches in gameplay mechanics, ensuring a smooth and unified multiplayer experience.

Acceptance Criteria

- The server must send a packet containing the enabled feature flags to all connected clients during the initial handshake process and whenever feature flags are updated.
- The packet must include all active feature flags, such as 'minecraft:vanilla', in a structured format that the client can parse and apply.
- Clients must correctly process the received feature flags and update their configurations to match the server's active features.
- The feature must integrate with the server's configuration management system, ensuring that any changes to feature flags are automatically reflected in the packets sent to clients.
- The synchronization process must be efficient, with minimal impact on server and client performance, even in scenarios with a large number of connected clients.
- The system must include error handling to manage cases where the client fails to process the feature flags packet, providing fallback behavior or appropriate error messages.
- The feature must be compatible with the latest version of the server and client software, ensuring seamless operation across different platforms and devices.
- The feature must be tested for edge cases, such as rapid changes to feature flags or network interruptions during synchronization, to ensure stability and reliability.
- The server must log the synchronization process, including the feature flags sent and any errors encountered, for debugging and monitoring purposes.
- The feature must adhere to security best practices, ensuring that only authorized clients receive the feature flags and that the data is transmitted securely.

Server Configuration Completion Notification

Description

The 'Server Configuration Completion Notification' feature in the 'Litsa Minecraft Cobol' application ensures that clients are promptly informed when the server configuration process is complete. This feature is designed to provide a seamless transition from server initialization to gameplay or other interactions. Once the server has successfully loaded all necessary configurations, it sends a dedicated packet to the connected client(s), signaling the completion of the setup process. This notification is tightly integrated with the server initialization workflow, ensuring that the client is updated in real-time without delays. By providing this feedback, the feature enhances user experience, reduces uncertainty, and ensures that players or administrators can proceed with their intended actions without unnecessary waiting or confusion.

Acceptance Criteria

- The server must send a 'configuration complete' packet to the client immediately after all configuration processes are successfully completed.
- The notification packet must include a unique identifier or flag to distinguish it from other server messages, ensuring clarity and preventing misinterpretation.
- The client must be able to receive and process the notification packet, triggering appropriate actions such as updating the user interface or enabling gameplay features.
- The feature must integrate seamlessly with the server initialization workflow, ensuring that the notification is sent only after all required configurations are validated and loaded.
- In cases where the server configuration fails or encounters errors, the notification must not be sent, and an appropriate error message or log must be generated for debugging purposes.
- The notification process must demonstrate high performance, with minimal latency between the completion of the configuration and the delivery of the packet to the client.
- The feature must be compatible with all supported Minecraft versions and client protocols, ensuring consistent behavior across different environments.
- The notification must be logged on the server side for auditing and debugging purposes, including a timestamp and the list of clients that received the notification.

- The feature must include error handling to manage scenarios where the client is disconnected or unresponsive during the notification process, ensuring the server remains stable.
- The implementation must adhere to security best practices, ensuring that the notification packet cannot be spoofed or tampered with during transmission.
- The feature must be tested across various deployment environments, including containerized setups, to ensure consistent behavior and reliability.
- The notification must be accessible to developers and administrators through server logs or debugging tools, providing insights into the server's state during initialization.

Tag Data Synchronization

Description

The 'Tag Data Synchronization' feature in the 'Litsa Minecraft Cobol' application ensures that clients always have the most up-to-date tag data for accurate and consistent gameplay interactions. Tags are used to group related items, blocks, or entities under a common identifier, enabling efficient gameplay mechanics such as crafting, block placement, and entity interactions. This feature works by sending a dedicated packet from the server to the client containing the latest tag data, including registry names, tag identifiers, and associated entries. The system is optimized for reuse, assuming that tag data does not change frequently, which minimizes unnecessary network traffic and improves performance. Additionally, the feature integrates seamlessly with the tag management system, ensuring that any updates to tags are automatically reflected in the synchronization process. This functionality is critical for maintaining consistency between the server and clients, especially in multiplayer environments where accurate tag data is essential for gameplay mechanics.

Acceptance Criteria

- The server must send a dedicated packet containing tag data to the client during the initial connection or when tag data is updated.
- The tag data packet must include registry names, tag identifiers, and associated entries, ensuring comprehensive synchronization.
- The feature must integrate with the tag management system to automatically detect and propagate changes to tag data.
- The system must optimize for reuse by caching tag data and only sending updates when changes occur, reducing unnecessary network traffic.
- Clients must correctly parse and apply the received tag data, ensuring accurate gameplay interactions such as crafting, block placement, and entity behavior.
- The synchronization process must demonstrate high performance, with minimal latency during data transmission and application.
- The feature must include error handling to manage potential issues such as packet loss or corrupted data, ensuring stability and providing fallback mechanisms.
- The tag data synchronization must be compatible with the latest version of the server and client, ensuring seamless operation across different environments.
- The feature must be tested across various scenarios, including large datasets and high player counts, to ensure reliability and scalability.
- The system must log synchronization events for debugging and monitoring purposes, providing insights into the synchronization process and any potential issues.

Block and World Updates

Block Update Notification

Description

The 'Block Update Notification' feature in the 'Litsa Minecraft Cobol' server ensures that any changes to blocks in the game world are accurately communicated to the client. When a block is modified (e.g., placed, destroyed, or updated), the server sends a packet containing the block's position and ID to the client. This ensures that the game world remains visually consistent and synchronized between the server and client. The feature is tightly integrated with the world update system, ensuring that all block changes are captured and transmitted in real-time. This functionality is critical for maintaining a seamless multiplayer experience, as it ensures that all players see the same state of the game world. Additionally, the feature is optimized for performance, minimizing the impact on server resources while handling large-scale updates efficiently.

Acceptance Criteria

- The server must send a block update packet to the client whenever a block is placed, destroyed, or updated in the game world.
- The block update packet must include the block's position (x, y, z coordinates) and its ID, ensuring accurate rendering on the client side.
- The feature must integrate with the world update system to ensure consistency and avoid missing any block changes.
- The client must correctly render the updated block state upon receiving the block update packet, ensuring visual synchronization with the server.
- The feature must handle multiple block updates efficiently, batching updates when necessary to optimize network performance.
- The system must support error handling to manage scenarios where block update packets fail to reach the client, ensuring stability and providing fallback mechanisms.
- The feature must be tested across different Minecraft versions supported by the server to ensure compatibility and consistent behavior.
- The block update notification system must demonstrate high performance, with minimal latency between the server detecting a block change and the client rendering the update.
- The feature must support edge cases, such as updates to special blocks (e.g., doors, beds, slabs) that require additional metadata for proper rendering.
- The implementation must adhere to the server's modular design principles, allowing for easy maintenance and future enhancements.
- The feature must include logging and debugging tools to help developers monitor block update packets and diagnose issues during development and runtime.

User Interface Customization

User Interface Screens

Description

The 'User Interface Screens' feature in the 'Litsa Minecraft Cobol' application provides a seamless mechanism for displaying specific UI screens, such as crafting tables, inventory management, or other interactive menus, to enhance user interaction and gameplay functionality. This feature works by sending precise instructions from the server to the client, triggering the client-side UI rendering system to open the appropriate screen. It ensures a smooth and responsive user experience by synchronizing server-side logic with client-side visual elements. The feature is designed to be modular, allowing easy integration with additional UI screens as new gameplay mechanics are introduced. It also supports customization and extensibility, enabling developers to define new UI workflows or modify existing ones. This feature is critical for maintaining an engaging and intuitive user experience in the game.

Acceptance Criteria

- The server must be able to send instructions to the client to open specific UI screens, such as crafting, inventory, or custom menus, without errors or delays.
- The client-side UI rendering system must correctly interpret the server's instructions and display the appropriate screen with all necessary elements (e.g., slots, buttons, labels) in their correct positions.
- The UI screens must be responsive and visually consistent across different devices and screen resolutions, ensuring a uniform user experience.
- The feature must support dynamic updates to UI screens, such as reflecting changes in inventory or crafting results in real-time as the user interacts with the interface.
- The system must handle edge cases, such as invalid or incomplete server instructions, by either displaying a fallback UI or providing an error message to the user.
- The feature must integrate seamlessly with other gameplay mechanics, such as inventory management, crafting systems, and player interactions, ensuring no conflicts or inconsistencies.
- The UI screens must adhere to accessibility standards, ensuring usability for players with disabilities (e.g., keyboard navigation, screen reader support).
- The feature must include a mechanism for developers to easily add or modify UI screens, with clear documentation and examples provided.
- The system must log any errors or issues related to UI screen rendering for debugging and troubleshooting purposes.
- The feature must be tested and verified to work across all supported Minecraft versions and platforms, ensuring compatibility and stability.
- The UI screens must support localization, allowing text and labels to be displayed in the player's preferred language.
- The feature must include performance optimizations to ensure that opening and interacting with UI screens does not introduce noticeable lag or performance degradation.

Experience Updates

Description

The 'Experience Updates' feature in the 'Litsa Minecraft Cobol' server ensures that players receive real-time feedback on their progress by dynamically updating their experience bar, level, and total experience points. This feature integrates seamlessly with the client-side UI, providing a visually intuitive representation of experience gains or losses. The server sends experience updates to the client whenever a player earns or spends experience points, ensuring that the displayed values are always accurate and synchronized. This feature enhances gameplay by offering immediate feedback on actions such as defeating mobs, mining, or using experience in crafting or enchanting. Additionally, it supports modular design principles, making it easy to extend or customize for future updates.

Acceptance Criteria

- The server must send real-time updates to the client whenever a player's experience points, level, or experience bar changes.
- The client-side UI must accurately display the updated experience bar, level, and total experience points without noticeable delay.
- The experience bar must visually reflect partial progress toward the next level, with smooth transitions for any changes.
- The feature must handle edge cases, such as when a player gains enough experience to level up multiple times in a single event, ensuring the level and experience bar are updated correctly.
- Experience updates must be triggered by all relevant in-game actions, including but not limited to defeating mobs, mining, smelting, and using experience in crafting or enchanting.
- The server must validate and synchronize experience data to prevent discrepancies between the server and client states.
- The feature must support modular integration, allowing developers to easily extend or modify the experience update logic for custom gameplay mechanics.
- The experience update system must be optimized for performance, ensuring minimal impact on server and client processing times, even during high-frequency updates.
- The feature must include error handling to manage potential issues, such as network latency or packet loss, ensuring the client receives accurate updates.

- The experience update system must be compatible with the latest version of the server and client, and it must be tested across different devices and operating systems for consistency.
- The feature must adhere to accessibility standards, ensuring that visual indicators of experience updates are clear and distinguishable for all players, including those with visual impairments.

Gameplay Mechanics

Player Command Handling

Description

The 'Player Command Handling' feature in the 'Litsa Minecraft Cobol' app is responsible for managing player commands such as sneaking and stopping sneaking. This feature ensures that player states are updated in real-time, providing a seamless and dynamic gameplay experience. When a player initiates a command (e.g., sneaking), the system processes the input, updates the player's state, and synchronizes it with other gameplay mechanics like movement and interaction. This integration allows for accurate representation of the player's current status in the game world, ensuring consistency across client and server interactions. The feature is designed to handle commands efficiently, even in multiplayer environments, and supports extensibility for future command additions.

Acceptance Criteria

- The system must detect and process player commands such as sneaking and stopping sneaking in real-time.
- Player state changes (e.g., entering or exiting the sneaking state) must be reflected immediately in the game world and synchronized with the server.
- The feature must integrate seamlessly with other gameplay mechanics, such as movement and interaction, ensuring that the player's current state is accurately represented during these activities.
- The sneaking state must affect gameplay mechanics appropriately, such as reducing movement speed or altering collision detection, as per game design specifications.
- The feature must support multiplayer environments, ensuring that player state changes are broadcasted to other connected players without noticeable delays.
- Commands must be processed efficiently, with minimal impact on server performance, even under high player load.
- The system must include error handling to manage invalid or unexpected command inputs, ensuring stability and providing appropriate feedback to the player.
- The feature must be compatible with the latest version of the app and tested across different devices and operating systems for consistency.
- The implementation must adhere to security best practices, ensuring that player commands cannot be exploited to gain unauthorized advantages or disrupt gameplay.
- The feature must include logging and debugging tools to assist developers in monitoring and troubleshooting command handling during development and live operation.

Item and Block Interaction

Description

The 'Item and Block Interaction' feature in the 'Litsa Minecraft Cobol' server processes player interactions with items and blocks, enabling complex gameplay mechanics. This feature determines the appropriate callback (item use or block interaction) based on the player's actions and the context of the interaction. It seamlessly integrates with the inventory system, world state, and player status to execute the correct behavior. For example, when a player uses a tool like a pickaxe on a block, the system triggers the appropriate block-breaking logic. Similarly, when a player interacts with an object like a crafting table, the system opens the crafting interface. This feature ensures that all interactions are contextually accurate and responsive, providing a smooth and immersive gameplay experience. It also supports custom behaviors for specific items and blocks, allowing for extensibility and customization.

Acceptance Criteria

- The system must correctly identify the context of a player's interaction, distinguishing between item use and block interaction.
- Item interactions must trigger the appropriate behavior, such as consuming an item, placing a block, or activating a tool-specific action (e.g., using a bucket to collect water).
- Block interactions must trigger the correct block-specific behavior, such as opening a door, activating a lever, or breaking a block with the appropriate tool.
- The feature must integrate with the inventory system to ensure that item usage updates the player's inventory correctly (e.g., reducing item count or removing the item if fully consumed).
- The feature must interact with the world state to ensure that block changes (e.g., breaking or placing blocks) are accurately reflected in the game world and synchronized with all connected clients.
- Player status (e.g., game mode, abilities) must be considered when determining interaction outcomes. For example, players in creative mode should not consume items when placing blocks.
- Custom behaviors for specific items and blocks must be supported, allowing developers to define unique interactions (e.g., custom tools or blocks with special properties).
- The system must handle edge cases, such as invalid interactions (e.g., trying to use an item on an invalid target) gracefully, providing appropriate feedback to the player.
- Interactions must be processed with minimal latency to ensure a responsive gameplay experience, even in multiplayer environments.

- The feature must include robust error handling to manage potential issues, such as invalid item data or corrupted block states, without crashing the server.
- The feature must be tested across different Minecraft versions supported by the server to ensure compatibility and consistent behavior.
- The system must log significant interaction events (e.g., block breaking, item usage) for debugging and analytics purposes, providing insights into player behavior and server performance.

Multiplayer Interaction

Swing Animation Broadcasting

Description

The 'Swing Animation Broadcasting' feature in the 'Litsa Minecraft Cobol' server ensures that player actions, such as swinging a tool or weapon, are visually represented to other players in the multiplayer environment. When a player performs a swing action, the server captures this event and broadcasts an animation packet to all connected players, excluding the initiating player. This feature integrates seamlessly with the player state and animation systems, ensuring that the animations are synchronized across all clients. By providing real-time visual feedback, this feature enhances the multiplayer experience, making interactions more immersive and intuitive. The implementation is optimized for performance, ensuring minimal latency and efficient packet handling even in high-player-count scenarios.

Acceptance Criteria

- The server must detect swing actions performed by a player and trigger the appropriate animation event.
- Animation packets must be sent to all connected players except the initiating player, ensuring that the initiating player does not receive redundant updates.
- The animation must be synchronized with the player state and other game systems to ensure consistency across all clients.
- The feature must handle edge cases, such as when a player disconnects during an animation broadcast or when a swing action is performed in rapid succession.
- The animation packets must be lightweight and optimized to minimize network bandwidth usage, ensuring smooth performance even in high-player-count environments.
- The feature must integrate with the server's packet handling system, ensuring that animation packets are correctly parsed and transmitted without errors.
- The swing animation must be visually accurate and match the initiating player's action, providing a consistent and immersive experience for other players.
- The feature must include error handling to manage potential issues, such as packet loss or desynchronization, and provide fallback mechanisms to maintain a stable multiplayer experience.
- The implementation must be tested across different client configurations and network conditions to ensure compatibility and reliability.
- The feature must adhere to the server's modular design principles, allowing for easy maintenance and future enhancements.

Inventory Management

Item Usage Handling

Description

The 'Item Usage Handling' feature in the 'Litsa Minecraft Cobol' server ensures that item interactions are processed accurately and consistently, particularly in survival mode. This feature manages the lifecycle of item usage, from the moment a player initiates an action (e.g., consuming food, using tools, or placing blocks) to the synchronization of inventory states and acknowledgment of the action. It integrates seamlessly with the inventory management system and gameplay mechanics to ensure that item behaviors align with game rules and player expectations. For example, when a player uses a consumable item, the server deducts the item from the inventory, applies the corresponding effect (e.g., restoring health or hunger), and sends a confirmation packet to the client. This ensures immersive gameplay by maintaining consistency between the server and client states. Additionally, the feature includes robust error handling to manage edge cases, such as invalid item usage or desynchronization, and provides feedback to the player when an action cannot be completed.

Acceptance Criteria

- The server must process item usage requests from clients and validate the action based on the current game state (e.g., survival mode rules, item availability).
- Inventory states must be updated accurately after an item is used, reflecting changes such as item consumption, durability reduction, or removal from the inventory.
- The server must send acknowledgment packets to the client after successfully processing an item usage request, ensuring synchronization between the client and server.
- For consumable items, the server must apply the appropriate effects (e.g., health restoration, hunger replenishment) and notify the client of the changes.
- For tools and weapons, the server must reduce durability appropriately and handle cases where the item breaks, removing it from the inventory if necessary.
- The feature must integrate with the inventory management system to ensure that item stacking, swapping, and other inventory operations are not disrupted by item usage.
- The server must handle invalid item usage requests gracefully, providing feedback to the client (e.g., 'Item cannot be used in this context') without crashing or desynchronizing the game state.
- The feature must support multiplayer environments, ensuring that item usage actions are broadcasted to other players when necessary (e.g., placing blocks, using potions).
- The server must log item usage actions for debugging and analytics purposes, including details such as player ID, item type, and action timestamp.
- The feature must be tested across different game modes (e.g., survival, creative) and scenarios to ensure consistent behavior and compatibility.
- The implementation must demonstrate high performance, processing item usage requests with minimal latency to maintain a smooth gameplay experience.
- The feature must include error handling for edge cases, such as network interruptions or corrupted item data, ensuring the server remains stable and responsive.
- The feature must adhere to the server's modular design principles, allowing for future extensions or modifications (e.g., adding new item types or behaviors) without significant refactoring.

Infrastructure Optimization

Optimized Multi-Stage Docker Build

Description

The 'Optimized Multi-Stage Docker Build' feature in the 'Litsa Minecraft Cobol' application ensures efficient and secure deployment by leveraging Docker's multi-stage build capabilities. This feature separates the build and runtime environments into distinct stages. The build stage includes all necessary tools, libraries, and dependencies required to compile the application, while the deploy stage contains only the compiled binary and essential runtime dependencies. By isolating these stages, the final Docker image is significantly smaller, reducing resource usage and improving deployment efficiency. Additionally, this approach minimizes security risks by excluding unnecessary build tools and files from the production image, ensuring a clean and lightweight runtime environment. This feature is particularly beneficial for maintaining a streamlined CI/CD pipeline and optimizing resource utilization in containerized environments.

Acceptance Criteria

- The Dockerfile must implement a multi-stage build process, with at least two stages: a build stage and a deploy stage.
- The build stage must include all necessary tools, libraries, and dependencies required to compile the application, such as GnuCOBOL, C++, and Makefile utilities.
- The deploy stage must only include the compiled binary and essential runtime dependencies, excluding all build tools and intermediate files.
- The final Docker image size must be significantly smaller than a single-stage build image, demonstrating the optimization achieved through the multi-stage process.
- The runtime image must be free of unnecessary files, such as source code, temporary build artifacts, and unused libraries, ensuring a clean and secure environment.
- The feature must be compatible with the application's cross-platform deployment requirements, ensuring the Docker image can run seamlessly on different operating systems and architectures.
- The multi-stage build process must be integrated into the CI/CD pipeline, ensuring automated and consistent image creation during the build and deployment phases.
- The Dockerfile must include comments and documentation explaining the purpose of each stage and the rationale behind the separation of build and runtime environments.
- The build process must be tested to ensure that the compiled binary functions correctly in the deploy stage, with no missing dependencies or runtime errors.
- The feature must support version-specific build optimizations, ensuring compatibility with different versions of the application and its dependencies.
- The Docker image must pass security scans to verify that no vulnerabilities or unnecessary components are present in the runtime environment.
- The multi-stage build process must be validated for performance, ensuring that the build and deployment times are optimized without compromising functionality.
- The feature must include error handling to provide clear feedback in case of build failures, such as missing dependencies or incorrect configurations.
- The final Docker image must be tested in a production-like environment to ensure it meets performance, security, and functionality requirements.

Signal Handling for Containerized Applications

Description

The 'Signal Handling for Containerized Applications' feature ensures the reliable operation of the application within containerized environments by properly managing process signals. This feature leverages Tini as the entry point in the Docker container to handle signals such as SIGTERM and SIGINT, ensuring that the application can gracefully shut down or restart when required. By preventing issues like orphaned processes or improper termination, this feature enhances the stability and maintainability of the application. Tini acts as a lightweight init system, forwarding signals to the main application process and cleaning up zombie processes, which is critical for long-running containerized applications. This feature is particularly useful in environments where containers are orchestrated using tools like Kubernetes or Docker Compose, as it ensures seamless integration with container lifecycle events.

Acceptance Criteria

- The Docker container must use Tini as the entry point, ensuring proper signal handling and process management.
- The application must gracefully shut down when receiving termination signals (e.g., SIGTERM, SIGINT), saving any necessary state and releasing resources.
- The feature must prevent orphaned processes by ensuring that all child processes are properly terminated when the container stops.
- The application must restart cleanly when a restart signal is received, reinitializing all necessary components without leaving residual processes or corrupted states.
- The implementation must be compatible with container orchestration tools like Kubernetes, ensuring that lifecycle events such as pod termination or scaling down are handled without errors.

- The feature must include logging to provide visibility into signal handling events, such as when a signal is received and how the application responds.
- The container must pass stress tests simulating rapid start-stop cycles to verify the robustness of signal handling and process cleanup.
- The feature must be tested across different operating systems and container runtimes (e.g., Docker, containerd) to ensure consistent behavior.
- The implementation must not introduce significant overhead to the container startup time or runtime performance.
- Documentation must be provided for developers and system administrators, explaining how the signal handling works, how to configure it, and how to troubleshoot common issues.

Data Integration

Automated Data Extraction from External Sources

Description

The 'Automated Data Extraction from External Sources' feature in the 'Litsa Minecraft Cobol' application streamlines the process of fetching and processing data from Mojang's server.jar file. This feature is designed to automate the download, extraction, and transformation of data into actionable reports and data files. Using a Makefile, the system downloads the server.jar file, executes Java-based data extraction scripts, and generates structured outputs such as reports and data files. These outputs are then seamlessly integrated into the final application build, enabling the application to provide data-driven insights and outputs. This feature ensures consistency, reduces manual effort, and enhances the reliability of data processing workflows.

Acceptance Criteria

- The Makefile must successfully automate the download of the Mojang server.jar file from the specified source URL.
- The system must validate the integrity of the downloaded server.jar file to ensure it is not corrupted or tampered with.
- Java-based scripts must extract relevant data from the server.jar file, including metadata, configuration details, and other required information.
- The extracted data must be processed and transformed into structured reports and data files in formats such as JSON, CSV, or XML, as required by the application.
- The generated reports and data files must be included in the final application build without requiring additional manual intervention.
- The workflow must handle errors gracefully, such as network failures during download or issues with the server.jar file, and provide meaningful error messages to the user.
- The feature must log all key steps in the workflow, including download, extraction, and report generation, for debugging and auditing purposes.
- The system must ensure compatibility with the latest version of the Mojang server.jar file and provide a mechanism for updating the workflow if the file structure changes.
- The feature must demonstrate high performance, completing the entire workflow (download, extraction, and report generation) within a reasonable time frame.
- The generated reports and data files must be verified for accuracy and completeness, ensuring they meet the application's data requirements.
- The feature must be tested across different environments to ensure consistent behavior and compatibility with the application's build process.
- The workflow must be documented clearly, including instructions for developers on how to trigger the Makefile and troubleshoot common issues.

Data-Driven Application Outputs

Description

The 'Data-Driven Application Outputs' feature in the 'Litsa Minecraft Cobol' application enables the generation of application-specific outputs by processing external data. This feature is designed to extract and process data from Mojang's server.jar, transforming it into meaningful reports and outputs that are utilized by the main application binary. The workflow involves parsing the server.jar to extract relevant data, applying transformations and validations, and generating structured outputs such as configuration files, analytics reports, or metadata summaries. These outputs are seamlessly integrated into the application's runtime environment, enhancing its functionality and providing actionable insights for users. This feature is essential for ensuring that the application remains data-driven, adaptable, and capable of leveraging external data sources to improve its operations.

Acceptance Criteria

- The feature must successfully extract data from Mojang's server.jar, ensuring compatibility with the latest version of the file format.
- The data extraction process must handle errors gracefully, providing meaningful error messages and logs in cases of invalid or corrupted input files.
- The application must validate the extracted data to ensure its integrity and correctness before generating outputs.
- The feature must support the generation of multiple output types, including configuration files, analytics reports, and metadata summaries, based on the processed data.
- Generated outputs must be structured and formatted according to predefined specifications, ensuring compatibility with the main application binary.
- The feature must integrate seamlessly with the main application binary, allowing the generated outputs to be consumed without additional manual intervention.
- The data processing workflow must demonstrate high performance, completing the extraction and output generation process within a reasonable time frame, even for large server.jar files.
- The feature must include robust logging and monitoring capabilities, enabling developers and administrators to track the data processing workflow and identify potential issues.

- The feature must adhere to security best practices, ensuring that sensitive data is not exposed or mishandled during the extraction and processing stages.
- The feature must be tested across different operating systems and environments to ensure consistent behavior and compatibility.
- The generated outputs must be version-controlled, allowing users to track changes and revert to previous versions if necessary.
- The feature must include documentation for developers, detailing the data extraction process, output formats, and integration points with the main application binary.

Quality Assurance

Test Automation Support

Description

The 'Test Automation Support' feature in the 'Litsa Minecraft Cobol' application ensures the reliability and functionality of the application by facilitating automated testing. This feature integrates seamlessly into the existing build process, leveraging the Makefile to compile and execute tests. By using the same build process as the main application, it guarantees consistency between the test environment and production builds. Developers and QA engineers can easily run tests to validate new changes, identify bugs, and ensure that the application meets quality standards before deployment. This feature significantly reduces the risk of introducing bugs into production and streamlines the testing workflow, making it an essential tool for maintaining application stability.

Acceptance Criteria

- The Makefile must include a dedicated target for compiling and running tests, ensuring that the test process is integrated into the build system.
- The test automation process must use the same build configuration and dependencies as the main application to ensure consistency between the test and production environments.
- Tests must be executed in an isolated environment to prevent interference with the main application or other processes.
- The test automation system must support running both unit tests and integration tests, covering critical components of the application.
- Test results must be displayed in a clear and concise format, including details about passed, failed, and skipped tests, to provide actionable feedback to developers and QA engineers.
- The system must handle errors gracefully, providing meaningful error messages and logs to help diagnose issues during the test execution process.
- The test automation process must be compatible with the application's cross-platform deployment strategy, ensuring that tests can be run on all supported platforms.
- The feature must support the addition of new test cases without requiring significant modifications to the existing test framework.
- The test automation system must demonstrate high performance, executing tests efficiently without introducing significant delays to the development workflow.
- The feature must include documentation explaining how to add, run, and interpret tests, ensuring that new developers and QA engineers can quickly adopt the testing process.

Build Automation

Version-Specific Build Optimization

Description

The 'Version-Specific Build Optimization' feature in the 'Litsa Minecraft Cobol' application enhances the build process by dynamically adapting to the installed version of GnuCOBOL. This feature ensures that the Makefile intelligently detects the GnuCOBOL version during the build process and adjusts compiler options accordingly. By leveraging advanced features available in newer versions or falling back to compatible options for older versions, this feature improves both performance and compatibility. It ensures that the build process is efficient, reliable, and tailored to the specific environment, reducing the risk of build failures and maximizing the use of available compiler capabilities. This feature is particularly useful for maintaining cross-version compatibility and optimizing the build process for diverse deployment environments.

Acceptance Criteria

- The Makefile must automatically detect the installed GnuCOBOL version during the build process without requiring manual intervention.
- The build process must dynamically adjust compiler flags and options based on the detected GnuCOBOL version, enabling advanced features for newer versions and providing fallback mechanisms for older versions.
- The feature must ensure that the build process completes successfully across all supported GnuCOBOL versions, maintaining compatibility and avoiding build errors.
- The Makefile must include clear and well-documented logic for version detection and corresponding compiler option adjustments, making it easy for developers to understand and modify if needed.
- The build process must log the detected GnuCOBOL version and the applied compiler options, providing transparency and aiding in debugging or troubleshooting.
- The feature must be tested across multiple GnuCOBOL versions to verify that the correct compiler options are applied and that the build process remains stable and efficient.
- The optimization logic must not introduce significant overhead to the build process, ensuring that build times remain reasonable.
- The feature must support extensibility, allowing future updates to include additional version-specific optimizations as new GnuCOBOL versions are released.
- The implementation must include error handling to manage scenarios where the GnuCOBOL version cannot be detected or is unsupported, providing clear feedback to the user and halting the build process gracefully.
- The feature must integrate seamlessly with the existing build system, including compatibility with other build automation features such as dependency management and multi-stage Docker builds.

Dependency Management for Source and Copybooks

Description

The 'Dependency Management for Source and Copybooks' feature in the 'Litsa Minecraft Cobol' application ensures efficient and accurate builds by automating the management of dependencies between source files and copybooks. This feature leverages a Makefile-based workflow to track and include dependencies in the build process. When a source file or copybook is modified, the Makefile automatically identifies the affected components and triggers the appropriate recompilation steps. This reduces build errors, improves maintainability, and ensures that the build process remains consistent and reliable. By streamlining dependency tracking, this feature minimizes manual intervention, enabling developers to focus on coding while ensuring that the build system remains robust and up-to-date.

Acceptance Criteria

- The Makefile must accurately track dependencies between source files and copybooks, ensuring that any changes to these files trigger the appropriate recompilation steps.
- The build process must automatically detect and include all relevant dependencies without requiring manual updates to the Makefile.
- The feature must support incremental builds, recompiling only the files affected by changes, to optimize build times.
- Error handling must be implemented to provide clear and actionable feedback when dependency issues are detected, such as missing or circular dependencies.
- The dependency management system must be compatible with the existing build environment, including GnuCOBOL, C++, and Makefile configurations.
- The feature must be tested across different platforms to ensure cross-platform compatibility and consistent behavior in the build process.
- Documentation must be provided to guide developers and build engineers on how the dependency management system works, including instructions for adding new dependencies or troubleshooting issues.
- The system must log dependency-related actions during the build process, such as files being recompiled, to aid in debugging and auditing.
- The feature must integrate seamlessly with version-specific build optimizations, ensuring that dependency tracking remains accurate across different Minecraft versions.
- The dependency management system must demonstrate high performance, with minimal overhead added to the build process, even for large projects with numerous dependencies.